ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ
ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS

ΣΧΟΛΗ ΕΠΙΣΤΗΜΩΝ & ΤΕΧΝΟΛΟΓΙΑΣ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ
SCHOOL OF INFORMATION SCIENCES & TECHNOLOGY
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
DEPARTMENT OF INFORMATICS



Mesh Decimation / Simplification via Weighted Importance Mapping

OptiX Repository: https://github.com/Heart-lex/MeshDecimation-OptiX
libigl Repository: https://github.com/Heart-lex/MeshDecimation-libigl

Students:

C. Chrysafi

A. A. Manetas

Athens, 2025

Table of Contents

1. Abstract 2
2. Introduction 3
3. Related Work 7
4. Methodology 10
    4.1 Data Preparation 10
    4.2 Visibility Analysis with NVIDIA OptiX 12
        4.2.1 Ray Generation 13
        4.2.2 Intersection Recording 13
        4.2.3 Perceptual Weight Computation 14
        4.2.4 Agent-Based Visibility Sampling 15
    4.3. Importance Map Construction 17
        4.3.1 Standardization 17
        4.3.2 Normalization 17
        4.3.3 Interpretation and Visualization 17
    4.4 Weighted Mesh Simplification with libigl 19
        4.4.1 Algorithm Overview 19
        4.4.2 Structural and Perceptual Constraints 20
        4.4.3 Material Tracking and Output 20
        4.4.4 Simplification Output 21
        4.5 Summary 21
5. Restrictions 22
    5.1. Geometry Requirements 22
    5.2. File Format Restrictions 23
    5.3. Computational Restrictions 23
    5.4. Algorithmic and Perceptual Constraints 25
    5.5. Scene and Data Limitations 25
6. Evaluations 27
    6.1 NVIDIA FLIP Comparisons 33
7. Conclusions 45
8. Definition of Terms 51
References 55

1. Abstract

This paper explores a visibility-driven approach to mesh decimation that integrates perceptual cues into the simplification process. Polygonal meshes are the fundamental representation of 3D geometry, but their complexity often becomes a bottleneck in real-time applications. High-resolution meshes provide visual realism but come at the expense of storage, processing, and rendering efficiency. This challenge has led to the development of mesh decimation techniques, which reduce geometric complexity while attempting to preserve the overall appearance of the model.

Traditional decimation strategies typically apply uniform simplification across all regions of a mesh, without considering the relative visual importance of different features. As a result, key areas such as silhouette edges, surface details, or frequently visible regions may be overly simplified, leading to noticeable degradation in visual quality. To address this limitation, our project introduces a visibility-driven importance mapping technique that prioritizes perceptually significant regions during the decimation process by employing NVIDIA OptiX, a high-performance ray-tracing framework, to gather visibility information which is then passed to a modified version of libigl's least-cost edge collapse algorithm.

The proposed pipeline demonstrates how perceptual information can inform mesh simplification to produce lightweight yet visually robust models. This approach has practical applications in scenarios where computational resources are limited but visual quality must be preserved, such as offline asset optimization for games, virtual environments, and simulation systems. By introducing visibility analysis into the decimation process, this work contributes to ongoing research in geometry processing and offers a practical tool for balancing efficiency with fidelity in 3D graphics.

2. Introduction

In recent years, 3D computer graphics have undergone rapid advancements, driven by the increasing demand for immersive experiences in gaming, animation, virtual reality (VR), film production, and simulation. At the center of the visual experiences lie polygonal meshes, which serve as the fundamental representation of 3D objects and environments. High-resolution meshes are capable of producing great levels of detail, but the increased graphical fidelity comes at a significantly high computational cost. Rendering highly detailed models almost always requires considerable processing power, making real-time performance in video games or VR simulations quite challenging.

Modern game engines like Unreal Engine 5 (UE5) [16] encourage game developers and designers alike to build detailed scenes with complex meshes, delivering a high level of realism. This realism though, can be overwhelming for hardware resources, leading to low frame rates, long loading times, and increased storage requirements.

A basic approach to addressing this challenge is mesh decimation, also known as mesh simplification. The purpose of decimation is to reduce the number of polygons in a mesh, while preserving its essential visual characteristics. This polygon reduction improves performance, given that it provides efficiency in storage, loading, and further processing of the 3D assets.

Traditional decimation algorithms often rely on the fact that all regions of a mesh are treated equally. While this simplifies the computational task, it risks losing critical visual features in important regions, such as frequently observed surfaces or fine geometric details.

To overcome this limitation, our paper explores and analyses a visibility-driven, importance-based decimation pipeline. Unlike the other algorithms, our approach evaluates the perceptual relevance of mesh regions and prioritizes the preservation of important features of the geometry. Specifically, we investigate how often different parts of a mesh are visible from an agent's perspective, which represents potential collected player movement data or pre-defined camera paths that would be used in real scenarios, within a scene created either directly in Unreal Engine 5 or externally through software tools such as Blender [8].

The process begins by exporting a scene from Unreal Engine 5 and analyzing it using NVIDIA OptiX, a high-performance ray tracing framework. OptiX is employed to cast rays throughout the environment and record how frequently each triangle is intersected. This ray-based visibility analysis forms the basis of an importance map, where hit triangles are considered visually significant, while those rarely visible or often occluded are marked as less important.

Once the importance values are established, we integrate them into a weighted mesh decimation process using libigl [1], an open-source geometry processing library. Unlike traditional uniform simplification, libigl allows us to assign weights to individual faces, enabling selective simplification. Regions with high importance values are preserved in greater detail, while areas with low importance undergo more aggressive simplification. The result is a significantly reduced mesh that maintains critical perceptual features while discarding redundant geometric complexity.



Figure 1.0.0: *Image representation of our paper's pipeline. The starting point is a scene created in either UE5 or Blender (or any other CAD tool with glTF and OBJ export capabilities), followed by the NVIDIA OptiX and libigl portions of the pipe, and the results can then be loaded back into the starting point programs of the pipeline (or again, any compatible tool).*

It is important to note that our work does not implement this system for real-time rendering within Unreal Engine 5. Instead, our contribution lies in the development of an offline pipeline that takes exported scenes, analyzes them with NVIDIA OptiX, and simplifies them using libigl. The outcome is an optimized version of the scene or mesh that is lighter, easier to store and process, and better suited for applications where efficiency is essential but perfect real-time fidelity is not required.

In summary, this paper addresses the problem of mesh complexity by combining visibility-driven importance mapping with weighted decimation. Our work demonstrates how ray-traced visibility information can guide the simplification process to produce optimized meshes that retain essential visual qualities. Although our implementation is offline, the method contributes to the broader field of mesh processing and opens potential applications in game development, simulation, and digital asset optimization, where balancing efficiency and visual quality is a persistent challenge.

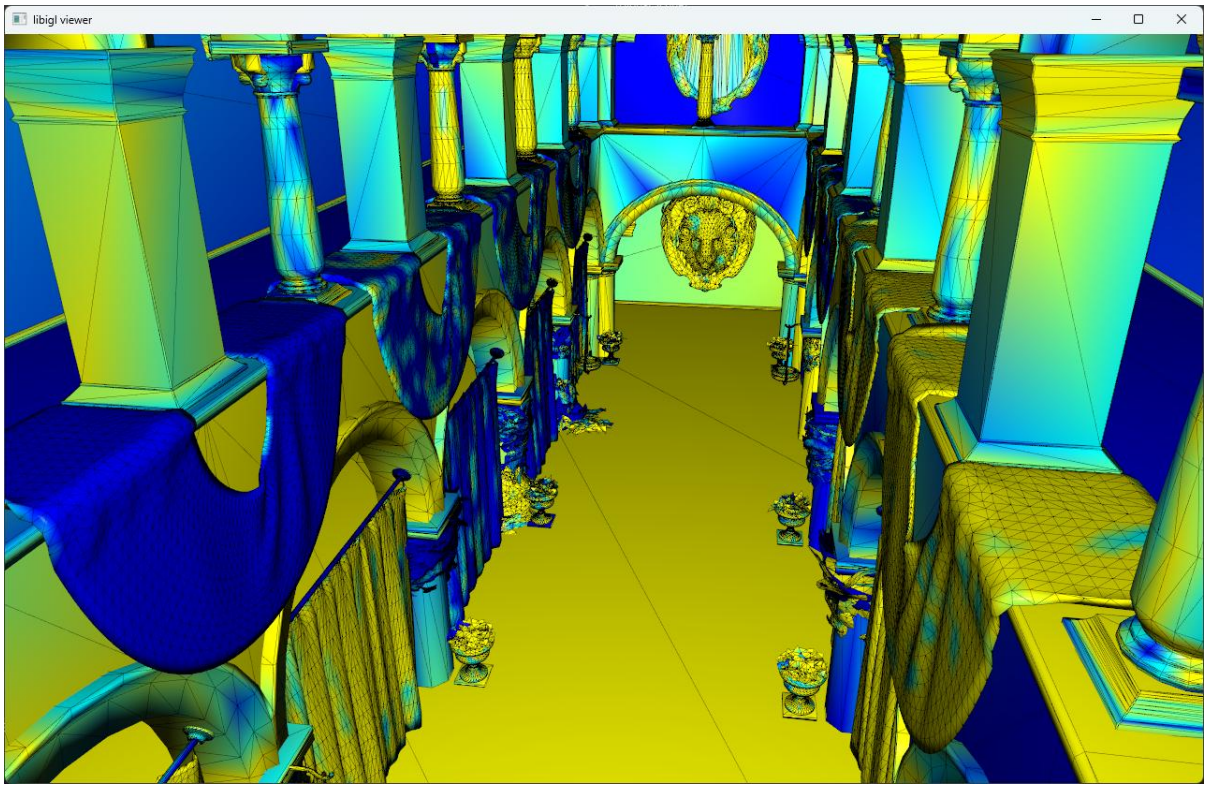


Figure 1.0.1: *One of the first weight map visualizations on UE5's Classic Sponza [19] scene. This particular scene ended up unused due to non-manifold geometry that was incompatible with the decimation algorithm.*

3. Related Work

In the field of mesh decimation research, the primary focus in recent years has been in utilizing local scene data such as lighting information, as well as topological properties of meshes in an attempt to alleviate the issues caused by mesh-agnostic decimation algorithms and ensure topological consistency.

In Xanthos Xanthopoulos' work *Evaluation of Illumination-Based Mesh Simplification* [4], the proposed method attempts to minimize the visible geometry's error in the final image via an importance heuristic that takes into account both direct and indirect lighting information. By tracing rays for multiple iterations over multiple passes from both the camera and the scene's various light sources, a per-primitive importance map is generated recursively on the GPU via the NVIDIA OptiX framework. The values generated in this stage represent the per-primitive traced importance and are transferred over to the CPU, where a unique mapping between the values and the respective vertices that they map to is created. This mapping is then forwarded to libigl where the decimation process can take place on either a per-mesh basis or on the 3D scene as a whole, using a decimation algorithm that is based on iterative edge collapses via a priority queue. In this stage, the heuristic is tuned to prefer the collapse of low importance areas first, simplifying in this way primitives that are of miniscule importance to the final image, thus minimizing the error rate between the original mesh and its decimated variant. A multi-pass simplification process to allow for large-scale decimation is also presented, where already decimated versions of the original meshes are used to perform the importance sampling on, which are then projected onto the original meshes. In its conclusion, one of the notes that is made is in regards to meshes with texture coordinates and how they could be integrated into the simplification process.
That subject matter is explored in Songrun Liu, Zachary Ferguson, Alec Jacobson and Yotam Gingold's *Seam erasure and seam-aware decoupling*

of shape from mesh resolution [5], where a suite of seam-aware processing techniques are proposed, taking heavy advantage of parameterized meshes' texture coordinate intricacies. Part of the suite is a seam-aware decimation algorithm that allows the reuse of a mesh's texture irregardless of its decimation level. Combined with the paper's findings in seam erasure and seam straightening, the paper's mesh decimator is able to create seam-free decimated variants of meshes without the need for them to be manually altered to achieve such a result. The driving algorithm behind this decimator is an extended version of Garland and Heckbert's QSlim method [7], with an edge-collapse cost metric that is calculated by first spreading out said metric from the mesh's faces onto its vertices, and subsequently to its edges. This way, the cost metric ensures that each time the edge that minimizes the error rate is the one that will be candidate for collapse, and this is repeated until the desired decimation level of the mesh is reached. The rest of the presented work dives into the parametric domain and minimizing the discontinuity artifacts caused by the existence of seams, which is not pertinent to the subject matter at hand.

In Peter Lindstrom and Grek Turk's proposed method of *Image-driven simplification* [6]**,** the notion of multi-view information gathering is employed as a means of gathering the necessary data to perform the simplification procedures without having to rely on a mesh's geometry, but rather by using an image metric. This metric is the root mean square (RMS) of the pixel-wise differences of image pairs that have been captured from multiple uniformly distributed camera positions from around the mesh. Taking into account the minimum bounding sphere of vertices that include the entirety of the mesh so as to capture it wholly, this ensures adequate coverage for the image samples taken. By treating these sets of images as a single image mosaic, the difference metrics can be computed based on it, giving the maximum coverage possible for the metric. As with most other decimation algorithms, the criteria for deciding whether an edge should be collapsed is decided by the edge with

the least cost, ensuring the visual differences are kept to a minimum. A significant difference with the edge-collapse algorithms used in other papers is that the heuristic used for vertex placement is geometry-based, while the edge cost function is image-based and decided by the aforementioned RMS metric. Accompanying this intricacy is a new, implementation-independent method for assigning texture coordinates to the vertices of the decimated mesh, enabling high levels of visual quality if texture appearance is accounted for. An important finding of this research is that by utilizing an image-based decimation procedure, invisible or entirely occluded detail that is of no value visually and would normally be retained in traditional decimation algorithms, is now aggressively decimated. This allows for external or internal detail that would never be seen (in the case of opaque geometry) to be of minimal cost in terms of resources. Considerations are also made about the way the human vision system can be exploited to take full advantage of the way humans process images, with similar hypotheses as in the real world.

Our work takes advantage of recent breakthroughs in the field of raytracing where CUDA-capable GPUs are now able to perform complex raytracing calculations in real-time, allowing for a ray-based solution to be possible. Many of the findings of the aforementioned papers are explored and utilized under a new perspective, that of traced importance.

4. Methodology

The proposed pipeline for visibility-driven mesh decimation consists of four primary stages: data preparation, visibility analysis, importance map construction, and weighted mesh simplification. In this section we provide a detailed explanation of the workflow, including the technical considerations and implementation details that shaped the methodology.

4.1 Data Preparation

As an initial stage of our work, we constructed a simple test level in *Unreal Engine 5 (UE5)*. The purpose of this environment was not to serve as a final case study, but rather to help us familiarize ourselves with the editor, experiment with simple tools, and validate the pipeline export process. This test scene became the foundation for our early OptiX and libigl experiments (Figure 1.1).

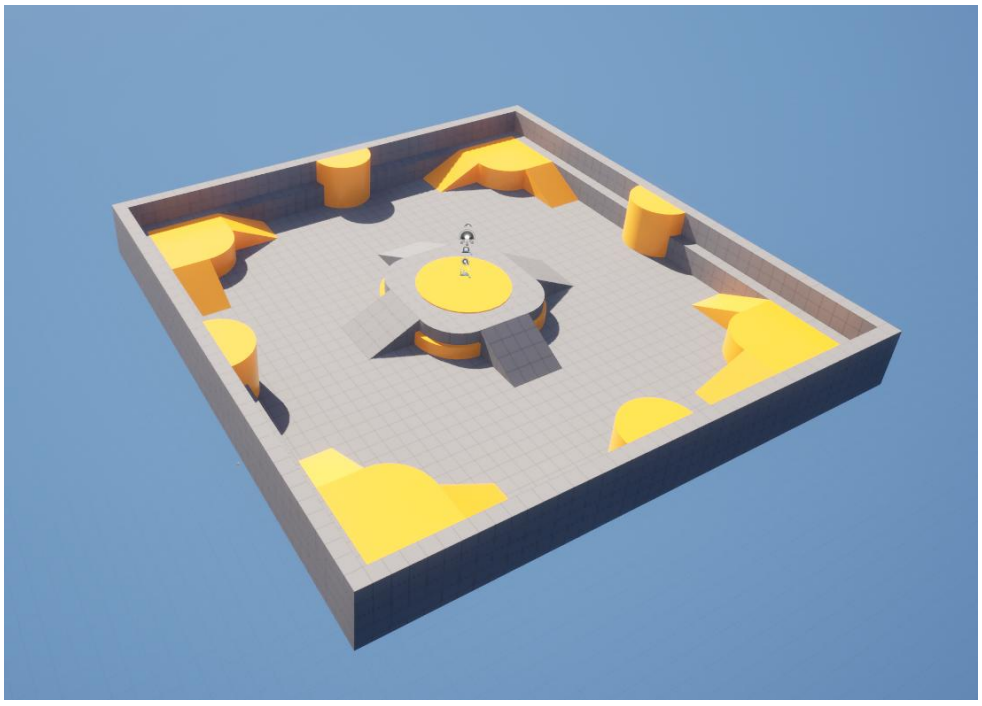


Figure 1.0.2: *The first prototype scene in Unreal Engine 5, used for preliminary pipeline testing.*

The exported UE5 assets are converted into two different formats, depending on the stage of the pipeline. For the visibility analysis stage, we use glTF (GL Transmission Format) [9]. This format is particularly well-suited for OptiX because it encodes complete scene information, including node hierarchies, transformations, and meshes, in a compact and GPU-friendly representation (Figure 1.2). By leveraging glTF, we maintain a coherent scene structure during ray casting and ensure that OptiX can efficiently create the necessary acceleration structures and subsequently traverse them.

For the simplification stage, the geometry is exported into the OBJ format, which is widely supported by libigl. OBJ provides a simple and explicit polygonal mesh representation without extra scene overhead. Since libigl requires direct access to per-vertex and per-face data, the OBJ format offers a clean and reliable input. This dual-format workflow, glTF for OptiX and OBJ for libigl, ensures that each tool operates on the mesh in the most efficient way, while maintaining consistency across stages through shared indexing and metadata.

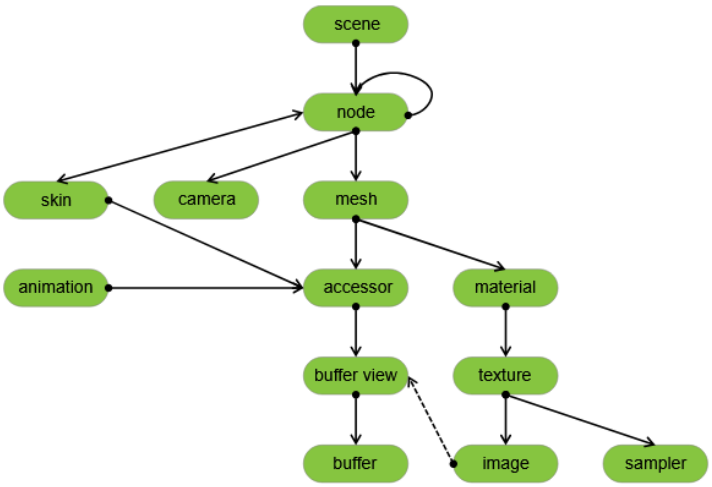


Figure 1.1: *Relations between top-level arrays in a glTF asset*

4.2 Visibility Analysis with NVIDIA OptiX

The core of the methodology lies in estimating how often each part of a mesh is visible. We utilize NVIDIA OptiX, a ray tracing framework that provides efficient intersection tests, making it well-suited for large-scale visibility analysis.

Before scaling to full scenes, we implemented a prototype ray casting mechanism to understand how OptiX records intersections, based on the optixPathTracer module of the OptiX SDK. The goal was to identify which faces of the meshes were hit by rays and, based on attributes such as distance, to assign preliminary weights to the triangles. After extensive testing and research, we succeeded in setting up a simple camera-driven ray casting system in OptiX, where rays were cast into our UE5 test level and collected per-face hit samples (Figure 1.3).

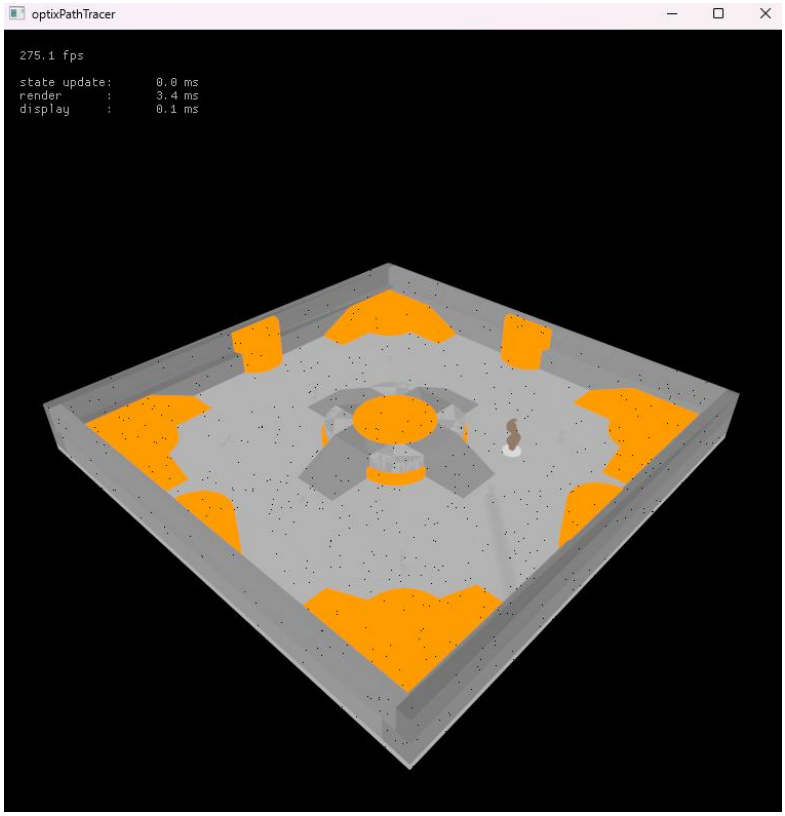


Figure 1.2.0: *The first ray casting test run for our first prototype scene.*

This prototype served as proof of concept for the visibility-driven weighting that would later guide simplification.

4.2.1 Ray Generation

Rays are cast from a fixed camera along the predefined paths. The camera samples its surrounding environment using a uniform distribution of rays across the field of view, simulating what a virtual observer would perceive. We parameterize the number of rays to balance accuracy with computational cost, where higher ray counts produce more stable visibility statistics but increase runtime.

Within the OptiX *__raygen__rg* program, each ray is generated from the pixel coordinates of a virtual image plane. A random subpixel jitter (*rnd(seed)*) ensures stochastic sampling, reducing aliasing and bias. For each ray, a corresponding payload structure (*RadiancePRD*) is initialized and traced through the scene.

4.2.2 Intersection Recording

Every time a ray intersects a triangle the *__closesthit__radiance* program is triggered. The global triangle index is computed as:

```
global_triangle_id = primitive_id + geometry_offset
```

This ensures that all meshes share a unified indexing space.

To mimic human visibility, only the first visible intersection along the ray is recorded - equivalent to what the eye would see in real-world occlusion scenarios. Each hit contributes a weighted visibility value to that triangle's cumulative importance score.

4.2.3 Perceptual Weight Computation

Each recorded intersection is assigned a perceptual weight derived from two biologically inspired components:

1. **Distance Weighting** – implemented in the *computeWeight()* device function:

   $$w_d = \frac{1}{\log_{10}(1 + d^2)},$$

   where d is a clamped safe distance based on the ray's distance to the hit point.

2. This logarithmic inverse-square falloff emphasizes nearby geometry and attenuates contributions from distant objects. Clamping prevents numerical instability for very close intersections.
3. **Foveatied Weighting** (w_f) – implemented in *calculateFoveatedWeight()*: This function models the drop in visual acuity from the human fovea outward. Using cosine similarity between the camera's forward direction and the ray direction, the algorithm assigns weights of 1.0, 0.51 to 0.99, and 0.1 to 0.5 for foveal (~5°), parafoveal (~20°), and peripheral (~45°) regions, respectively.

The final per-hit weight is defined as:

$$w_i = log(1 + w_d \cdot w_f)$$

and accumulated in global GPU memory:

```
params.d_triangle_weights[global_triangle_idx] += log(1 +
foveated_weight * computeWeight(distance));

params.d_triangle_not_occluded_hits_count[global_triangle_idx] += 1ul;

params.d_triangle_total_hits_count[global_triangle_idx] += 1ul;
```

Only rays with unobstructed visibility increment the “not occluded” counter, while all rays increment the total count, allowing per-triangle visibility ratios to be computed later.

4.2.4 Agent-Based Visibility Sampling

A secondary ray generation program, *__raygen__agent*, emulates a *virtual agent* performing a scan of its surroundings in place of real player or camera positional data. Four rays in four different directions (0°, 90°, 180°, 270°) are launched per step, and the distances to visible geometry in each direction are recorded.



Figure 1.2.1: *An early prototype of the OptiX agent walkthrough on UE5’s Classic Sponza scene. Triangle indexing had not been properly resolved yet, hence the erroneous textures.*

These measurements guide the agent’s movement, who picks the longest direction to traverse to every time a distance threshold to an occluding piece of geometry is reached. When this threshold is reached, the agent turns 180° away from the blockage and then at a random direction within a ±90° radius. Then, it checks which of the four rays has the longest distance, and travels towards that direction by 20% of said distance. This process is repeated for a number of pre-defined maximum steps.

This agent-driven visibility analysis bridges the gap between raw geometric sampling and perception-aware scene understanding.

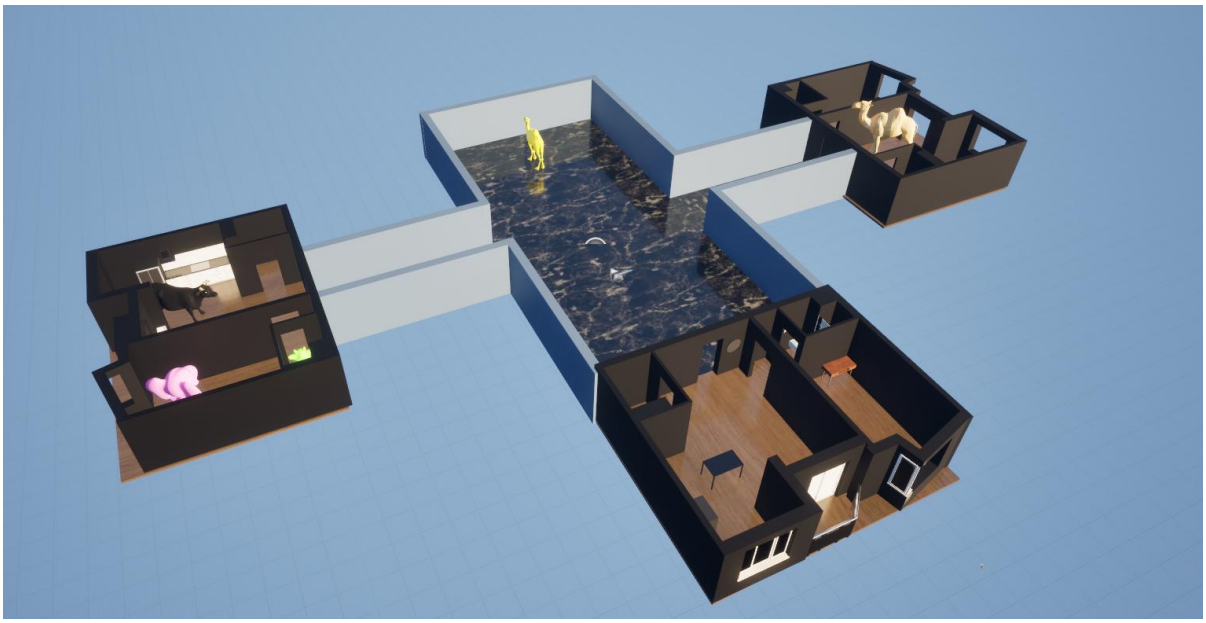


Figure 1.3: *Final prototype scene in Unreal Engine 5 (UE5), showing the environment explored by the agent during visibility sampling*

4.2.5 Exporting Visibility Data

After the ray tracing pass completes, all triangle weights are copied back from the GPU (device) to the CPU (host):

```
cudaMemcpyAsync(reinterpret_cast<void*>(state.params.h_triangle_weights), reinterpret_cast<void*>(state.params.d_triangle_weights), state.params.scene_triangle_count * sizeof(float), cudaMemcpyDeviceToHost);
```

This asynchronous memory copy procedure is repeated for the total hit counts as well as the total occluded hit counts, which are used in the importance map construction.
The weights are normalized to [0, 1] and exported as a TXT file. Each line corresponds to one triangle, storing its computed perceptual importance value. This file becomes the importance map used in the next stage.

4.3. Importance Map Construction

Once intersection data is collected, it must be translated into an importance map that guides simplification.

4.3.1 Standardization

Before normalization, we apply standardization to stabilize the distribution of hit frequencies. This step prevents excessively high or low values from dominating and ensures a balanced contribution across different regions of the mesh.

4.3.2 Normalization

Raw hit counts vary depending on the total number of rays cast. To allow consistent weighting across different meshes or experiments, hit counts are normalized into a 0 to 1 scale, where 1 represents the most frequently observed triangles and 0 represents triangles that were never visible.

4.3.3 Interpretation and Visualization

The resulting normalized values express the perceptual visibility of each triangle. Regions frequently visible to the agent or camera have high importance, while occluded or distant regions have low values. The importance map can also be visualized as a color-coded heatmap overlay in libigl, providing intuitive visual feedback (Figure 1.5).

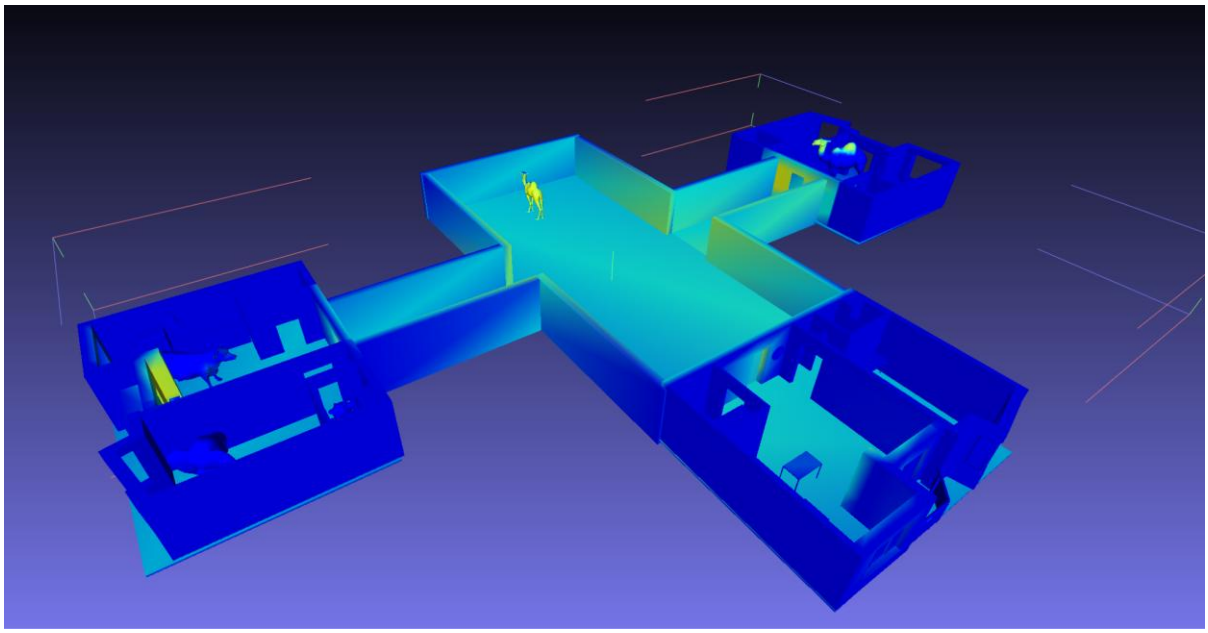


Figure 1.4.0: *Per-triangle importance visualization in heatmap view, highlighting frequently visible surfaces.*

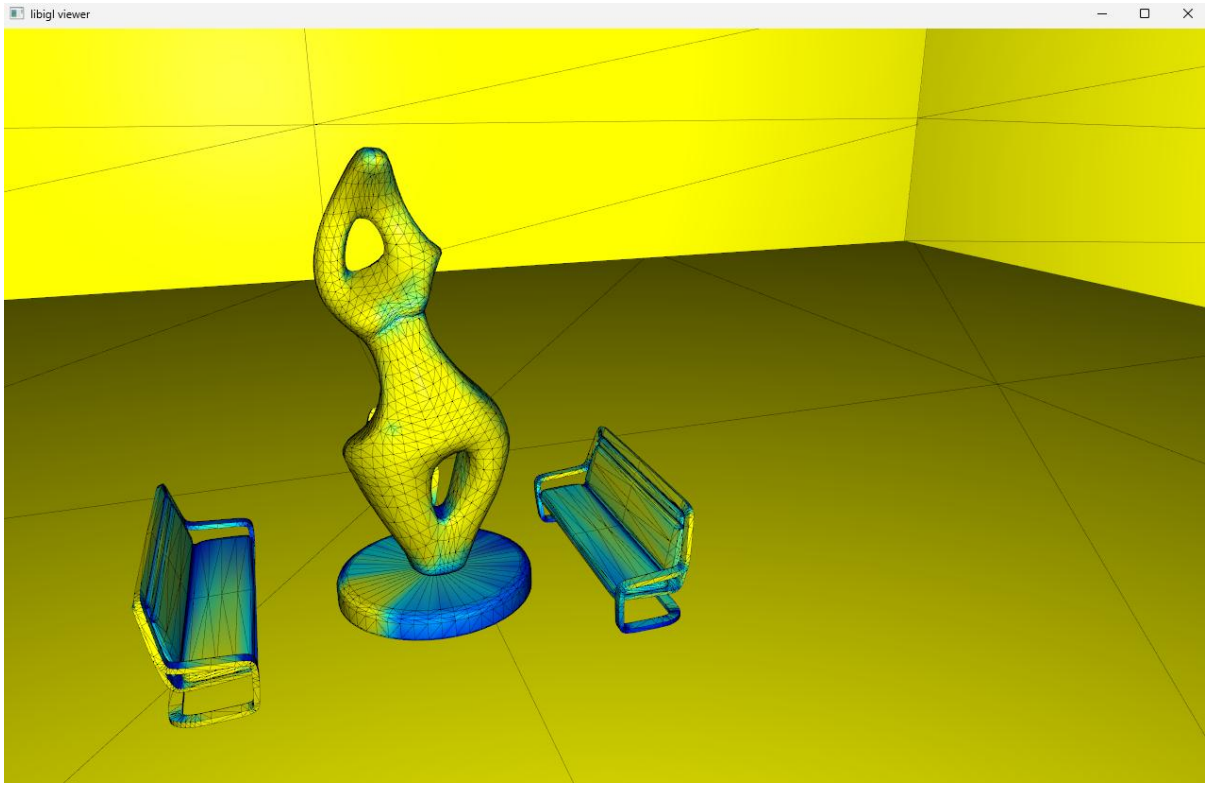


Figure 1.4.1: First run in libigl of our *per-triangle importance visualization in heatmap view of the first prototype scene.*

4.4 Weighted Mesh Simplification with libigl

With the importance-weighted map established, we integrate it into libigl, an open-source geometry processing library offering robust edge-collapse simplification.

4.4.1 Algorithm Overview

We extend libigl's least-cost edge-collapse method (Figure 1.6) by embedding perceptual weights into its cost function. Edges adjacent to high-importance faces receive higher costs, discouraging their collapse, while edges in low-importance regions are removed earlier.

This ensures that visually salient geometry, such as frequently viewed faces, silhouettes, or near-camera features, remains intact.

This simplification follows the sequence:

1. Compute all edges and adjacency (*igl::edge_flaps*).
2. Load weight table that has been generated from the OptiX pipeline for this mesh / scene.
3. For each edge, compute geometric cost and candidate collapse point (*shortest_edge_and_midpoint*).
4. Modify the cost based on local importance weights.
5. Store each computed edge into a min-heap priority queue based on its cost.
6. Collapse the least costly edge (*collapse_least_cost_edge*) while respecting boundary and material constraints.

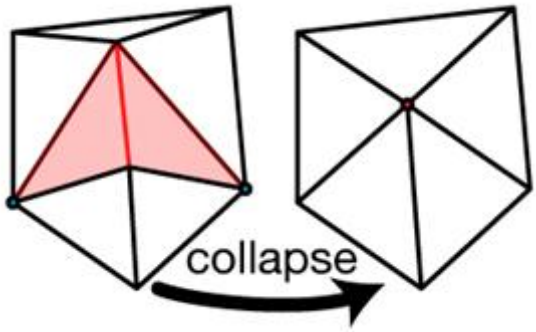


***Figure 1.5**: The edge collapse operation merges two adjacent vertices (highlighted in red), removing the associated edge and replacing the affected faces with a new vertex, thereby reducing mesh complexity.*

4.4.2 Structural and Perceptual Constraints

Custom logic ensures structural integrity:

- Boundary and sharp edges are preserved
- Long structural edges and horizontal/vertical planes (floors and walls) are protected.

These checks are handled by a custom function *isStructurallyImportant()*, which prevents undesirable topological degradation. Low-weight regions that are visually insignificant may collapse aggressively.

4.4.3 Material Tracking and Output

Material continuity is preserved through a callback that updates per-face materials after each collapse (*update_materials*). This ensures that multi-material objects remain visually consistent after simplification. Finally, the reduced mesh is written to disk using *writeOBJWithMaterialsAndUVs()* preserving geometry, texture coordinates, normals, and MTL references.

4.4.4 Simplification Output

The simplification process can run iteratively with different decimation factors, producing a hierarchy of meshes suitable for multiple Levels of Detail (LOD).

The user may export the mesh at any intermediate stage, enabling interactive exploration of visual quality versus triangle count.

4.5 Summary

The final pipeline integrates GPU-accelerated perceptual analysis with CPU-based geometric optimization:

Stage	Domain	Core Functionality	Output
Data Presentation	UE5/Blender → glTF/OBJ	Scene export, format conversion	Structured scene data
Visibility Analysis	OptiX (GPU)	Ray tracing, distance + foveated weighting	Per-triangle viability stats
Importance Map Construction	OptiX (CPU)	Normalization, Standardization	Importance map file
Weighted Simplification	libigl (CPU)	Edge-collapse with perceptual weighting	Simplified LOD meshes

In addition, our implementation provides the ability to save intermediate meshes at any stage of the decimation process. Rather than committing to a fixed reduction ratio, the algorithm runs iteratively until it is stopped, allowing the user to export the mesh at arbitrary levels of simplification. This flexible control makes it possible to experiment interactively with different polygon target counts, compare visual quality across multiple versions, and adapt the output to the requirements of different rendering contexts.

5. Restrictions

The proposed visibility-driven mesh decimation pipeline, while effectively preserving perceptually important surfaces, operates under a number of restrictions and assumptions that define its applicability and performance. These limitations stem from both technical constraints of the tools used (Unreal Engine 5, NVIDIA OptiX, and libigl) and the design decisions made during the development of the methodology. Understanding these restrictions is essential to correctly interpret the results and identify directions for future improvement.

5.1. Geometry Requirements

A primary requirement of the pipeline is that all input geometries must be manifolded. A manifold mesh ensures that each edge is shared by exactly two faces, allowing for well defined surface continuity. Non-manifold geometries can lead to incorrect visibility analysis, invalid normal calculations, or instability in the simplification process.

If the imported geometry contains a small number of non-manifold edges, these must be corrected prior to processing. This can be achieved using mesh-repair tools such as Blender's 3D Printing ToolKit, which provides automated detections and corrections for the non-manifolded edges. Ensuring manifold topology is particularly important for accurate ray-triangle intersection tests within NVIDIA OptiX and for maintaining consistent edge-collapse behavior in libigl.

Additionally, before exporting meshes for analysis, all geometry must be joined into a single unified object in Blender. If multiple objects remain unjoined, the importance distribution can become inconsistent, since parts of the geometry may be traced as separate entities. This prevents the correct assignment of per-face importance weights, especially in cases where objects overlap or share close spatial boundaries.

5.2. File Format Restrictions

Due to compatibility and stability considerations, the system relies on specific file formats at different stages of the pipeline.

- *glTF* (.gltf) is used during the visibility analysis phase, as it maintains scene hierarchy, transformations, and material properties in a GPU-friendly structure suitable for acceleration structures in OptiX, taking advantage of the framework's built-in helper functions (*sutil*). Additionally, this is the only file format that is supported by both OptiX and Unreal Engine 5.
- *OBJ* (.obj) is used during the simplification stage, as it provides a straightforward and explicit polygonal representation compatible with libigl's input routines. This also happens to be the most flexible file format supported by libigl.

Other file formats (such as FBX, STL, PLY, and OFF) were found to alter face indexing or introduce discrepancies in vertex ordering. These inconsistencies interfere with the mapping between importance weights and triangle indices, resulting in incorrect simplification behavior. Thus, the workflow requires OBJ input for libigl to ensure that per-face correspondence between the visibility data and geometric structure is maintained.

5.3. Computational Restrictions

The proposed pipeline operates offline and is not yet optimized for real-time performance. While NVIDIA OptiX provides GPU acceleration for ray tracing, the visibility analysis process remains computationally demanding, particularly when casting millions of rays across high-resolution scenes. The accuracy and granularity of the importance map depend directly on the number of rays sampled, introducing a fundamental trade-off between computational cost and statistical stability. Higher sampling densities yield more precise importance

estimation but significantly increase processing time and memory requirements. It should also be stated that RTX-capable graphics cards are a must, as NVIDIA OptiX requires GPUs with CUDA compute capabilities in order to execute properly.

Although the system is designed for static geometry and lighting conditions, it incorporates a moving agent within the environment. This agent, which follows predefined or simulated trajectories, dynamically collects visibility data along its path. By allowing the observer's viewpoint to move, the method captures a more representative distribution of visibility that closely resembles realistic camera or player behavior in interactive scenarios. Nonetheless, since both geometry and lighting remain static during analysis, the resulting importance map does not yet account for dynamic scene changes, such as moving objects, deformable meshes, or time-varying illumination.

Due to the decimation algorithm's serial nature and lack of parallel execution of its main logic, a full run on a scene containing approximately 120,000 triangles takes around 80 minutes to complete, with a triangle reduction ratio that reaches 90%. While the results end up being satisfying and in line with the expectations one would have based on the produced heat maps, the total runtime could be massively improved. By further optimizing data structures, implementing better edge collapse checks, parallelizing parts of the main thread, and improving upon libigl's built-in decimation routines (which the creators' own comments seem to signify that there is a lot of room for optimization in), future research could achieve speeds that are but a fraction of the current runtime.

Because of these constraints, the current implementation is best suited for offline preprocessing rather than real-time execution. However, the underlying principles and data collected through this pipeline provide a valuable foundation for developing real-time visibility-aware mesh decimation in the future. With further optimization, the same importance-driven logic could be integrated into modern rendering

engines to adaptively simplify geometry based on live visibility feedback, potentially enabling on-the-fly mesh optimization guided by actual player movement or camera behavior during gameplay.

5.4. Algorithmic and Perceptual Constraints

The decimation process is based on a modified least-cost edge collapse algorithm. While the integration of perceptual weighting improves feature preservation, the algorithm still approximates surface error using local geometric properties and may occasionally over-simplify subtle features not captured by visibility analysis. Moreover, the importance of weighting relies solely on visibility frequency and does not yet incorporate other perceptual factors such as lighting contrast, texture saliency, or motion relevance. This information could be gathered during the OptiX part of the pipeline, by leveraging a more advanced pathtracer such as NVIDIA's GLTF Renderer [20], which utilizes advanced BRDFs and BSDFs and more complex per-ray data.

In addition, occlusion handling is simplified: only the first intersection per ray is recorded, simulating single-layer visibility. This is sufficient for most applications but does not capture complex visual phenomena such as translucency or secondary reflections. For more advanced perceptual modeling, future work could incorporate multi-bounce ray tracing or light-driven weighting.

5.5. Scene and Data Limitations

The scenes used in this study were deliberately simple to ensure controlled testing. As such, the performance of the system on extremely large or highly complex production scenes has not been fully validated.

Scenes containing large amounts of instanced geometry, displacement surfaces, or non-uniform scaling may require further adaptation of the import and preprocessing pipeline.

Lighting and material data from UE5 are not currently utilized during the visibility analysis. Although OptiX supports physically-based rendering (PBR) and material shading, this project focuses entirely on geometric visibility, not photometric appearance. This limits the perceptual realism of the importance map but simplifies computational requirements.

Summary

In summary, the visibility-driven mesh decimation pipeline demonstrates promising results but remains subject to several technical and practical restrictions. Proper data preparation (manifold geometry, unified objects), consistent file formats (glTF and OBJ), and controlled camera setups are essential for successful operation. Computational limitations and simplified perceptual modeling constrain its current use to offline, static scenes. Nevertheless, these restrictions define a clear foundation for future improvements toward real-time, adaptive, and multi-modal mesh simplification.

6. Evaluations

This section presents the results of the visibility-driven mesh decimation process. It compares the original and simplified meshes to evaluate how effectively the proposed method reduces polygon count while preserving visual quality. Quantitative data and visual comparisons using NVIDIA FLIP are included to demonstrate the system's performance, highlighting the achieved reduction ratios and the preservation of the perceptually important regions.

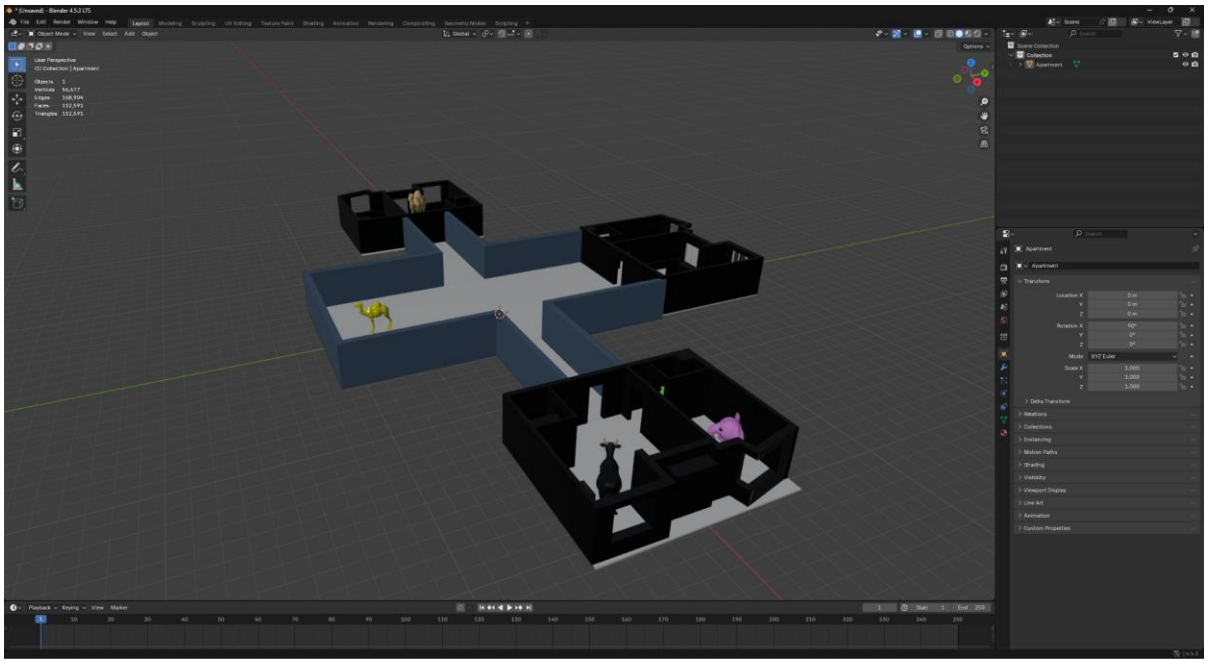


Figure 1.6.1: *Original mesh (0% reduction, 112,591 triangles).*

The figure above illustrates the original, unaltered mesh prior to any decimation. Composed of 112,591 triangles, it represents the full-resolution geometry of the scene as created in Blender before simplification.

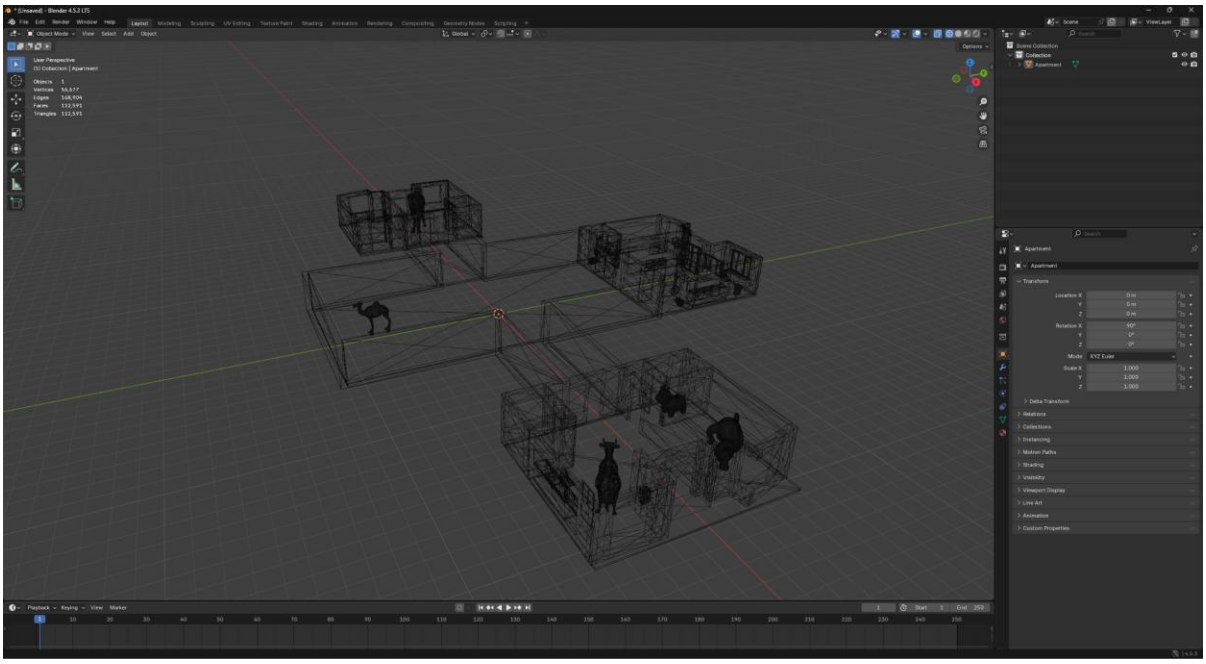


Figure 1.6.2: *Original mesh (0% reduction, 112,591 triangles, wireframe).*

The wireframe visualization of the original mesh highlights the dense polygonal distribution across all surfaces. It reveals the high geometric complexity of the unprocessed model, which serves as the baseline for evaluating the effectiveness of the simplification process

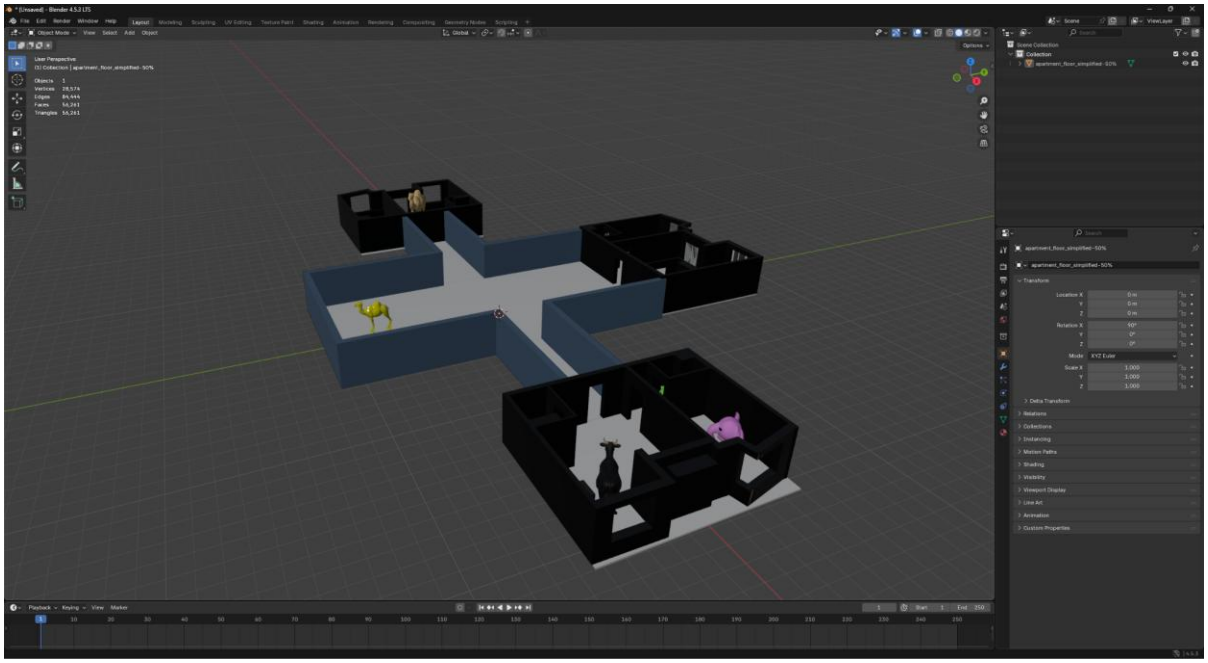


Figure 1.7.1: *Decimated mesh (50% reduction, 56,261 triangles).*

At 50% reduction, the decimated model demonstrates a substantial decrease in edge count while maintaining the overall shape and silhouette of the scene. Significant perceptual regions remain intact, proving that the weighting based algorithm effectively preserves visually important details.

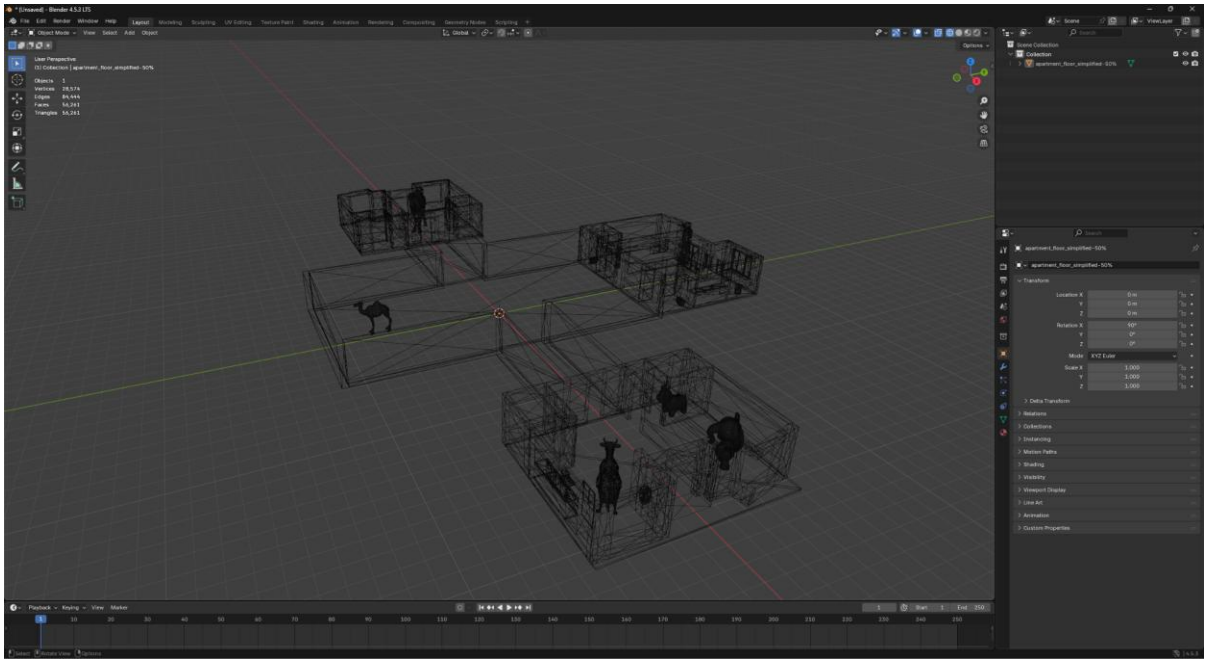


Figure 1.7.2: *Decimated mesh (50% reduction, 56,261 triangles, wireframe).*

The wireframe of the 50% decimated scene provides a clear insight into how simplification is distributed across the entire scene. Although large-scale topological changes are not immediately visible at the 50% reduction level, subtle reductions can be observed in less perceptually relevant regions (*i.e. the right room in the corner*). This proves the algorithm's ability to maintain structural fidelity while preparing for a larger scale of decimation. For this reason, the following example at 85% reduction is included to better illustrate the full extent of the geometric simplification process.

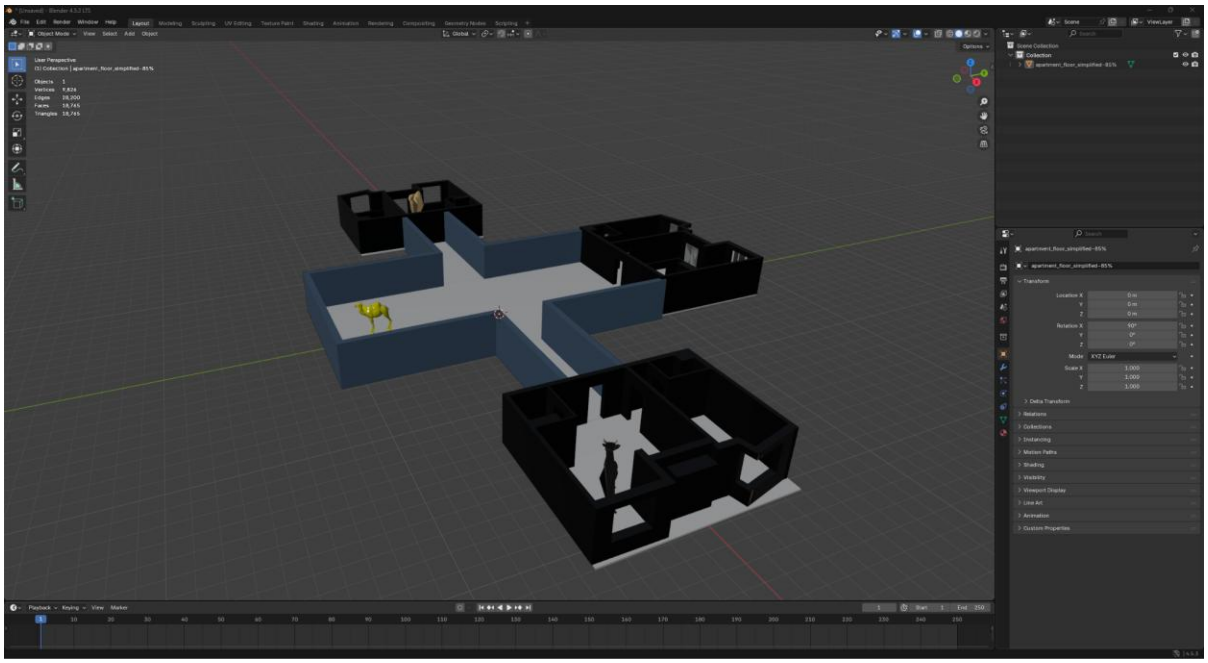


Figure 1.8.1: *Decimated mesh (85% reduction, 18,765 triangles).*

At 85% reduction, the decimation process produces a visibly lighter geometry while preserving the perceptually relevant structure of the scene. The elephant toy in the lower room is almost entirely removed, as it remained unseen by the agent. The rear part of the cow is mostly simplified, while its front remains intact. The large camel's lower section, fully occluded by the wall, is reduced considerably, while the smaller camel exhibits partial simplification on its front and right sides, as seen from the agent's POV, with the rest of its geometry preserved.

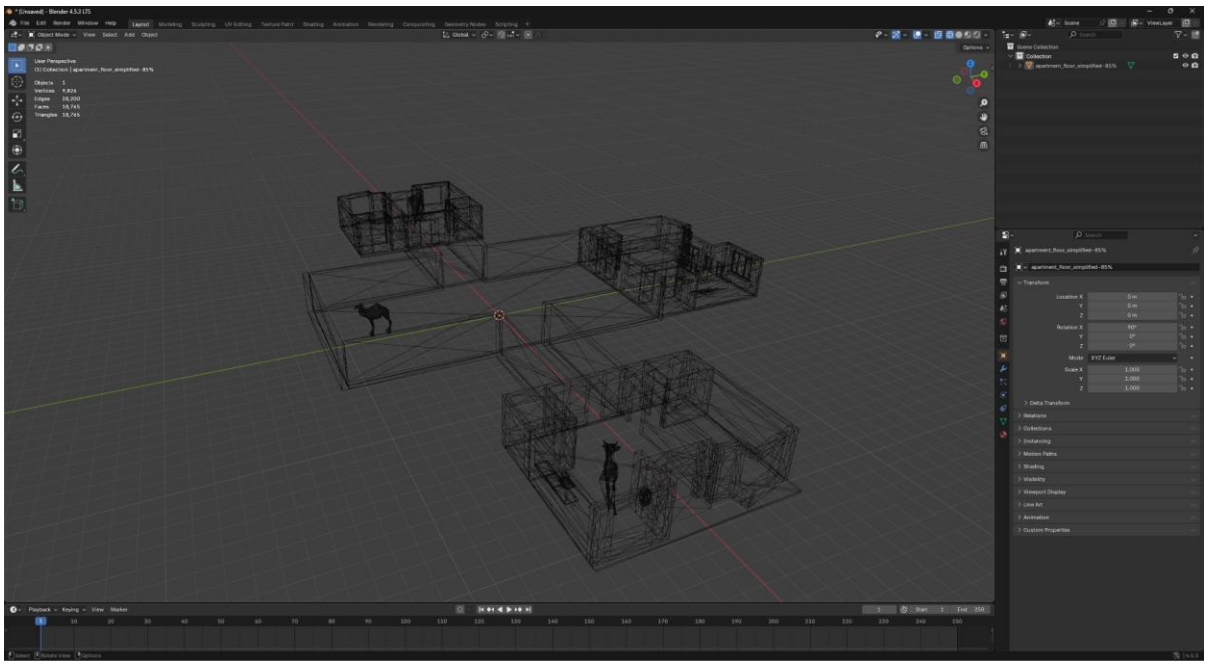


Figure 1.8.2: *Decimated mesh (85% reduction, 18,765 triangles, wireframe).*

In the wireframe view, the above observations can be seen clearly. Regions most exposed to the agent's viewpoint remain concentrated, while occluded and low-visibility areas display coarse and sparse topology. These results illustrate how our visibility-driven decimation selectively removes unseen or low-importance faces, while keeping the scene's overall form and perceptual clarity.

6.1 NVIDIA FLIP Comparisons

Following the evaluations of differently scaled decimated scenes, we use NVIDIA FLIP to visualize the differences in the simplified meshes from the agent's point of view. In these comparisons, the highlighted regions indicate high differences, where geometry has been altered or heavily simplified, while darker areas show little to no differences, representing parts of the mesh that remain intact. The screenshots that the comparisons are performed on were captured in Unreal Engine 5.

As the agent's path was mostly confined within the starting hallway of the scene, which also happened to be the agent's starting point, a large portion of the importance sampling was performed here. As such, we decided it would be an ideal starting point to showcase the visual differences of the scene's topology. Part of our procedure involved placing cameras in positions that were captured during the OptiX portion of the pipeline, so as to have a 1 to 1 depiction of the agent's point of view and what a potential player or camera actor would see or capture.

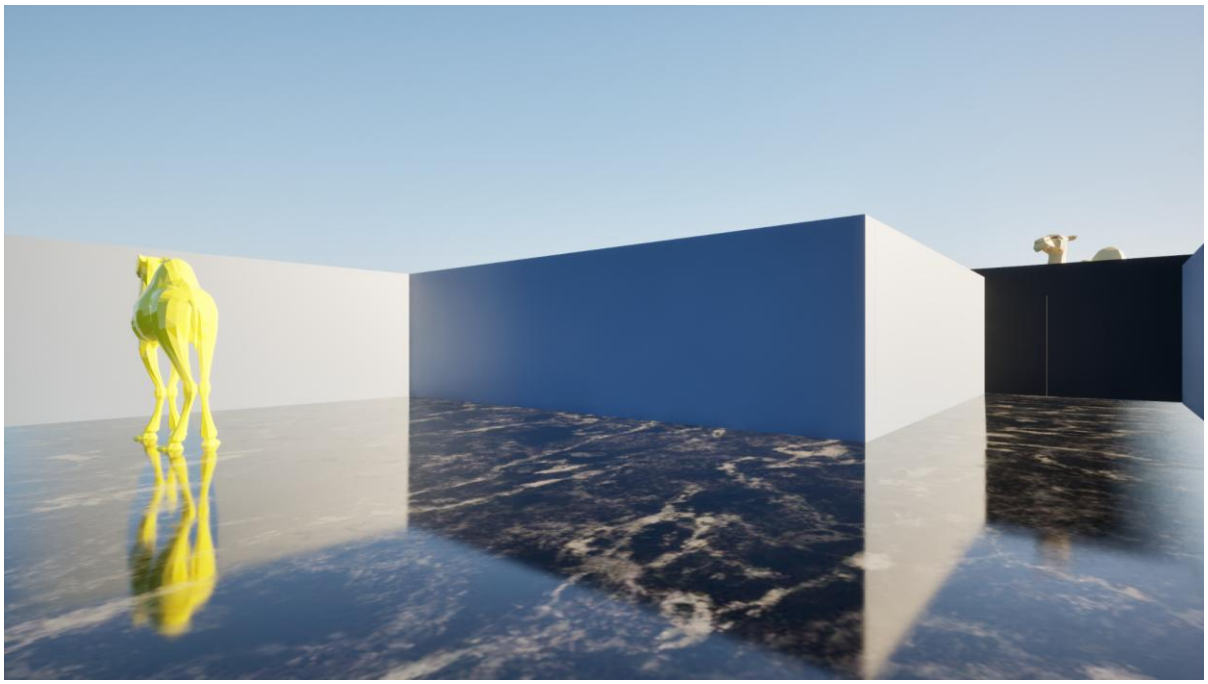


Figure 1.9.1: *Original scene, hallway, agent's point of view (POV).*

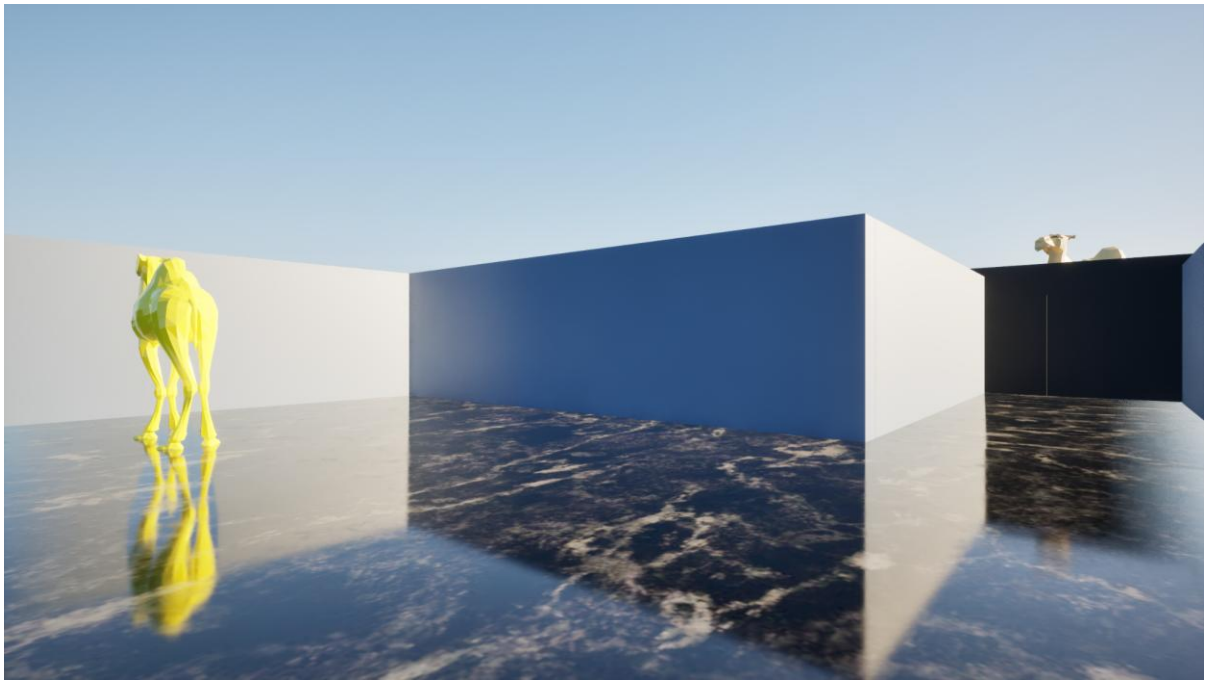


Figure 1.9.2: *Decimated scene, hallway, agent's point of view (POV).*



Figure 1.9.3: *Difference evaluation between the agent's POV in the original scene and the decimated scene, generated by NVIDIA FLIP at 67ppd (pixels per degree). The slight shimmer artifact on the bottom right is due to dithering of the scene's reflections due to Lumen's low-res surface cache.*

As seen by the results, the difference primarily involves the silhouettes of affected meshes, while the main geometry is left unaffected. By taking into consideration the human visual system's favorability of center of mass in terms of foveated viewing, this means that the differences are negligible to the naked eye.

To further showcase the degree of decimation, images were also captured from angles that the agent was not able to see in its exploration of the scene.

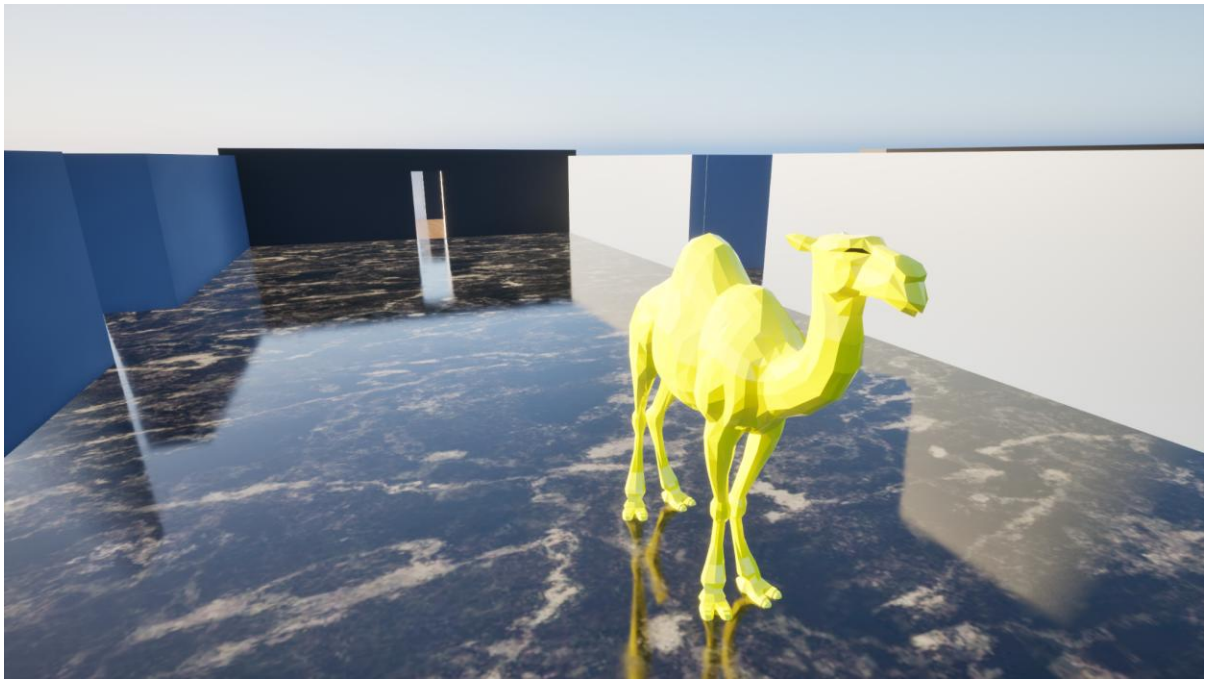


Figure 1.10.1: *Free-camera view from an angle that the agent did not see (original scene).*

From this point of view the mesh of the camel appears complete, while our invisible agent is located exactly behind it. When wandering around the scene the agent never actually gets to see the camel from the angle we see in Figure 1.10.1. That's why below we present the exact same location after the decimation process.

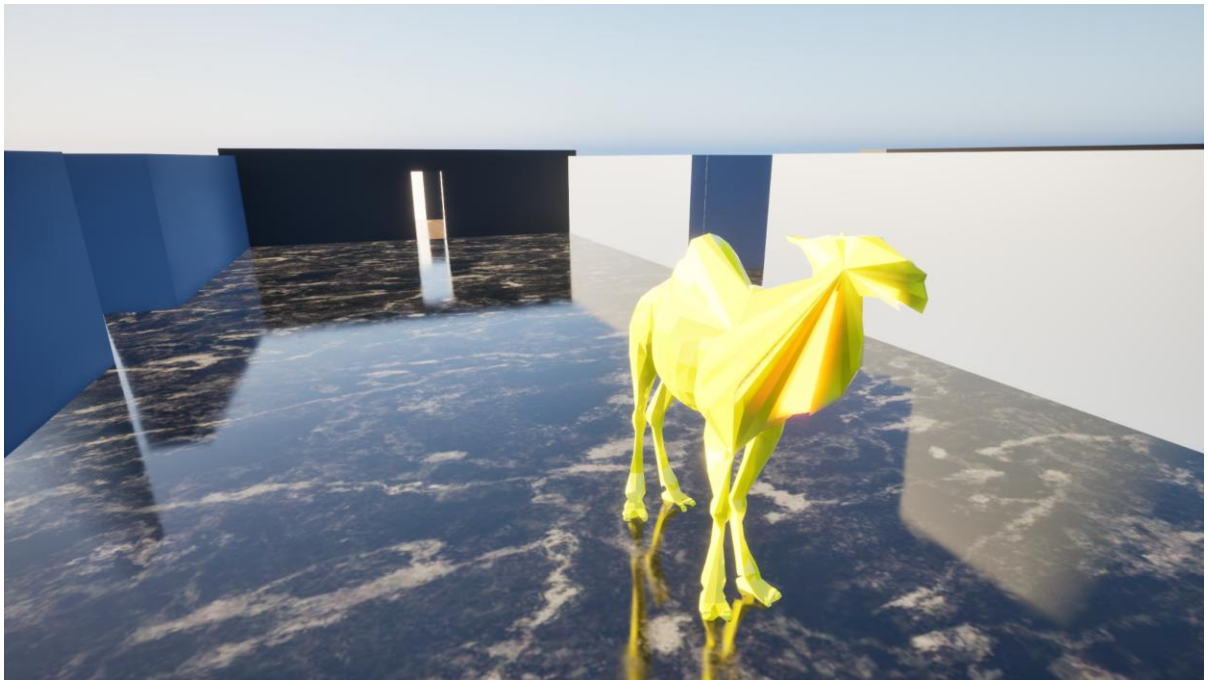


Figure 1.10.2: *Free-camera view from an angle that the agent did not see (decimated scene).*

Clearly, in Figure 1.10.2, we can observe that the background remains largely unchanged, as does the rear region of the camel's model. In contrast, the front upper regions of the mesh have been almost entirely decimated, since the agent never viewed the model from this particular perspective. Consequently, the visibility weights assigned to these upper areas were significantly lower compared to those of the rear section. This explains why the only visibly affected part of the camel is the one facing the current camera view.

Now, viewing another room of the same scene from a free-camera POV:

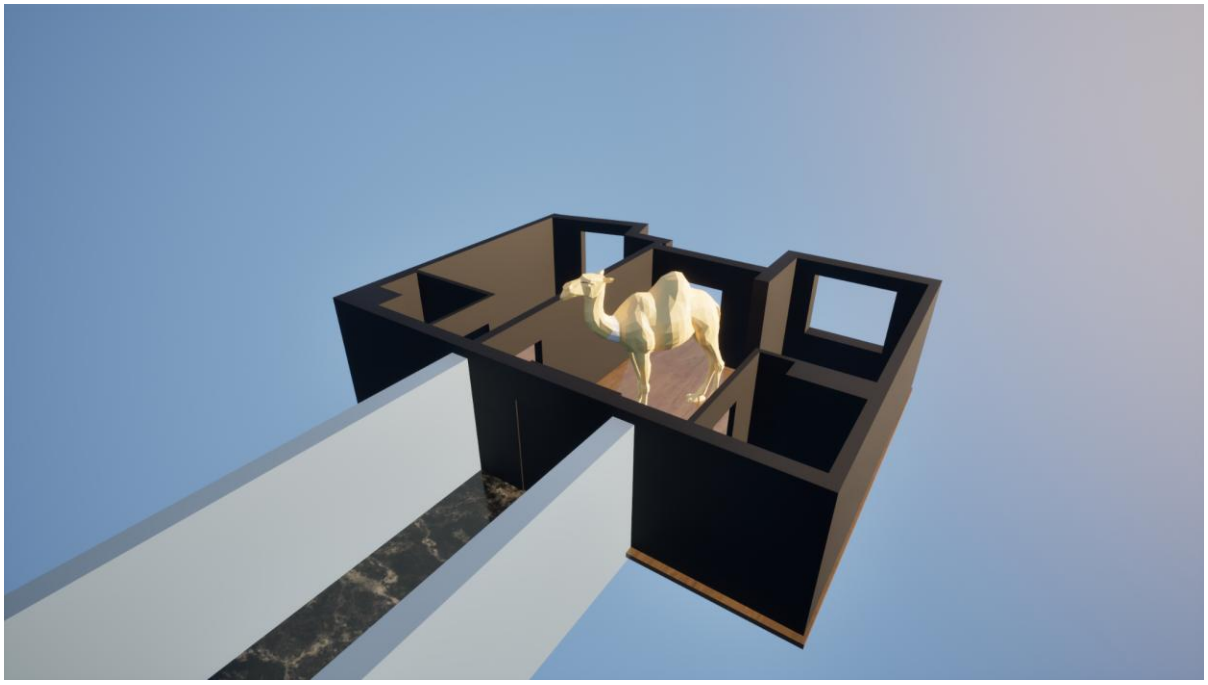


Figure 1.11.1: *Free-camera view of the obstructed mesh (original scene).*

In this room, the lower part of the camel is never visible from the point of view of the agent, since it is fully occluded by the wall. Only the head of the camel and portions of its upper part such as its hump are visible. This view serves as a clear demonstration that the pipeline effectively performs its intended task, fully removing the lower, unseen parts of this particular part of the scene once decimation is complete.

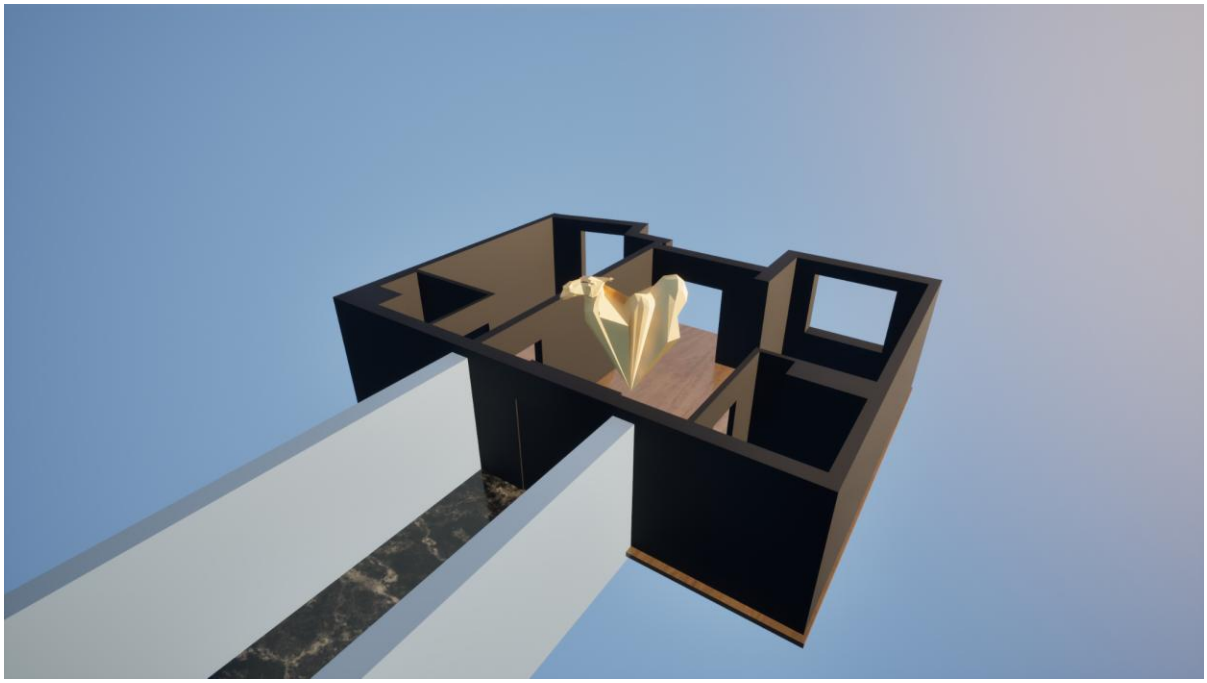


Figure 1.11.2: *Free-camera view of the obstructed mesh (decimated scene).*

After the decimation routine's completion, it becomes evident that the upper portions of the mesh remain almost entirely unchanged, as this region was the one most frequently observed by the agent. Consequently, the importance weights in this area are significantly higher, prompting the algorithm to preserve it during simplification.

In contrast, the remainder of the mesh, previously noted as being fully occluded by the surrounding walls, was assigned lower visibility values and thus underwent more aggressive decimation, as can be seen by the end result.

In the next example, the toy room makes its appearance, which is populated by three different large animal toys. Below, the image of the room from the agent's point of view is given:



Figure 1.12.1: *Original scene, toy room (bull), agent's point of view (POV).*

From this viewpoint, only a limited portion of the cow's mesh remains visible within the room, while the rest of its geometry, along with several surrounding toys, lies completely outside the camera's field of view.

Based on the visibility-driven nature of the proposed algorithm, we would therefore expect the rear section of the cow and the occluded toys to undergo significant simplification. Since these areas were never observed by the agent during the visibility sampling phase, their corresponding importance weights are expected to be low, leading the decimation process to remove or heavily reduce their geometric detail.



Figure 1.12.2: *Decimated scene, toy room (bull), agent's point of view (POV).*

After the decimation process is completed, the scene appears largely unchanged from the agent's point of view. However, despite the absence of any perceptible difference to the human eye, a substantial portion of the underlying geometry has been removed.

Below, we present a comparative example using visual difference images produced by NVIDIA FLIP, which clearly highlight the decimated areas of the mesh.



Figure 1.12.3: *Difference evaluation between the agent's POV in the original scene and the decimated scene, generated by NVIDIA FLIP.*

As previously noted, although the human eye cannot easily detect any differences, the image reveals that, when viewed from the same angle, the regions of the mesh that have been decimated correspond primarily to small protruding edges and minor geometric details.

To further illustrate this behavior, we will now examine the same scene from an alternative viewpoint, outside of the agent's field of vision, to analyze how the decimation process affects areas that were never directly observed.

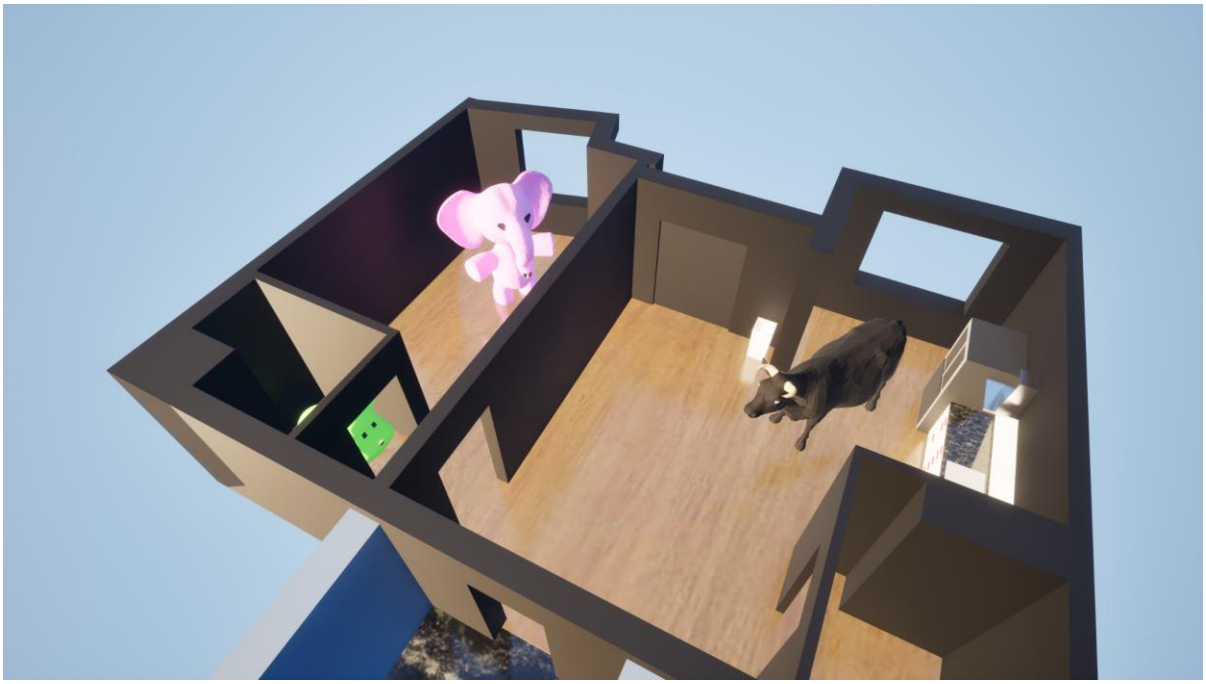


Figure 1.13.1: *Free-camera view of toy room (original scene).*

This perspective reveals portions of the scene that lie outside the agent's traversal path, showcasing all the toys within the room in their original, fully detailed geometric form. From the agent's field of view, however, only a small portion of the cow's frontal mesh is ever visible, while its rear section, and the remaining toys scattered throughout the room, remain entirely unobserved. Consequently, these unseen regions receive low or negligible visibility weights during analysis.

Below, the same scene is presented from the identical camera angle after the decimation process has been applied.

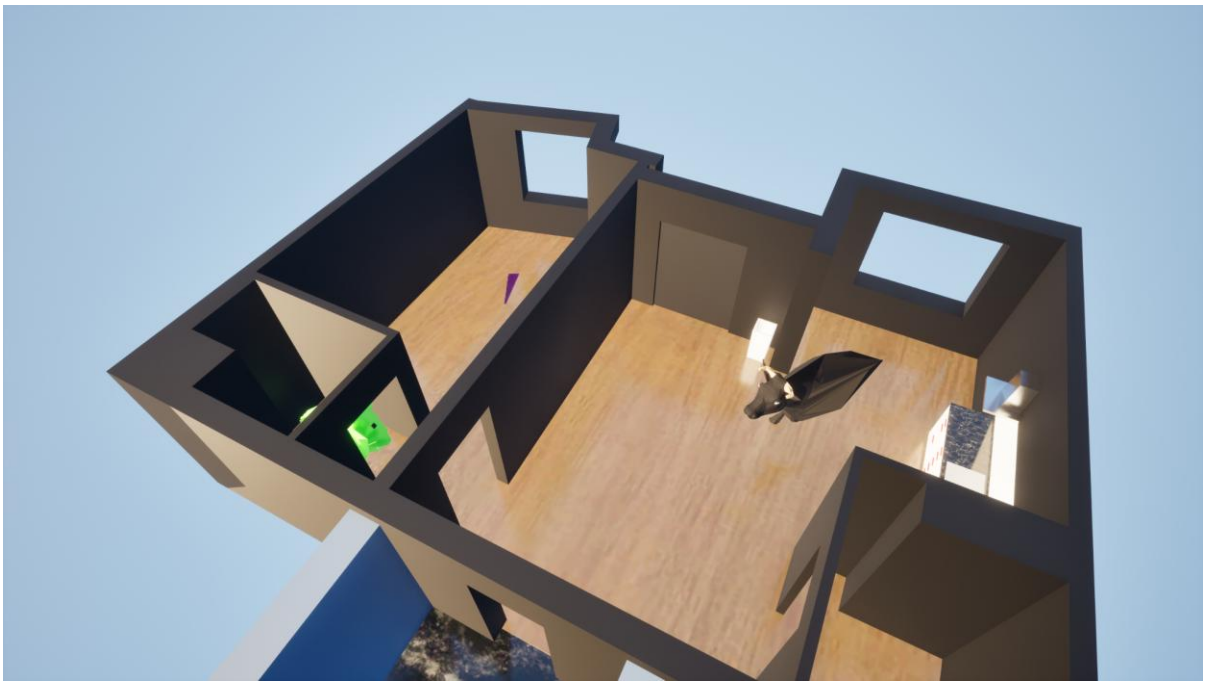


Figure 1.13.2: *Free-camera view from viewpoint that the agent did not see (decimated scene).*

After the decimation process, it becomes evident that the two occluded toys have been heavily simplified, along with the rear section of the cow model. This confirms that the algorithm correctly assigned low importance weights to regions that were never observed from the agent's point of view.

From the prior analysis, we can conclude that there are negligible differences between the original version of the scene and its decimated variant, when taking into account the agent's original point of view and traversal path. Supporting this are the image differences produced by the NVIDIA FLIP tool, which validate the fact that only small portions of the meshes' silhouette and sharp, boundary details are affected.

7. Conclusions

The work presented in this paper explored the problem of perceptual decimation in 3D environments, focusing on how visibility-driven analysis can guide the selective simplification of meshes. The objective was to design and implement a perceptual decimation system that leverages ray-traced visibility data to preserve geometrically and perceptually significant areas of a scene, while efficiently reducing the computational cost of rendering non-salient regions. Through the integration of NVIDIA OptiX for visibility evaluation and libigl for mesh processing, a hybrid pipeline was developed that demonstrates how perceptually informed geometry reduction can become both practical and scalable.

This study began with the recognition that traditional geometric simplification techniques often treat all parts of a 3D model as equally important. While such methods can effectively reduce vertex count, they disregard the human visual system's non-uniform sensitivity to detail. Humans do not perceive all parts of an image equally; attention naturally focuses on high-contrast regions, motion areas, and objects near the foveal center. Thus, a purely geometric decimation process can lead to perceptually sub-optimal results, where resources are spent preserving invisible or low-salience geometry while removing visually important features. By introducing a perceptual weighting step informed by actual scene visibility, the presented system aims to bridge this gap between mathematical optimization and perceptual realism.

From a technical standpoint, the project's primary contribution lies in the seamless coupling of GPU-accelerated visibility evaluation with CPU-based simplification. NVIDIA OptiX was used to generate dense visibility data via ray tracing from a set of camera viewpoints.

This data captures how often each triangle in the scene is visible from user-defined perspectives, effectively representing a measure of perceptual exposure. The visibility scores were then mapped onto mesh faces and stored in a TXT file, enabling libigl to use them as weighting factors during decimation. By assigning higher weights to frequently visible or perceptually important regions, the decimation process becomes adaptive to the way a human observer would experience the scene.

The results demonstrated that visibility-guided simplification achieves a balanced compromise between geometric efficiency and visual fidelity. In controlled tests, the perceptual decimation pipeline produced models with up to 50–85% reduction in polygon count while maintaining almost indistinguishable appearance from the original when viewed from typical camera positions. These outcomes suggest that perceptual criteria can outperform purely geometric ones in contexts where realism and performance must coexist such as virtual reality, games, or architectural walkthroughs. More importantly, they show that perceptual awareness can be integrated into traditional graphics pipelines without excessive computational overhead or radical redesign of rendering systems.

Beyond performance metrics, the conceptual significance of this work lies in its reframing of what mesh simplification represents. The algorithm does not indiscriminately remove geometry but rather makes context-aware decisions that respect how users see and interpret visual information. This reflects a broader shift in computer graphics from algorithm-centric optimization toward human-centric perception models. The presented system embodies this transition by coupling physically accurate rendering techniques (ray tracing) with perceptually grounded simplification logic.

An important insight that emerged during experimentation was that visibility data alone, though powerful, does not fully capture perceptual relevance. Visibility indicates how often a surface is seen, but not how much it contributes to visual salience. For example, a background wall may be visible from every viewpoint but contain little perceptual weight compared to a small, high-contrast object in motion. This opens the door for future work that incorporates additional perceptual features such as luminance contrast, shading gradients, or motion vectors to build a richer importance map. The integration of such cues could enable a more holistic perceptual decimation framework that adapts dynamically to lighting, movement, and user attention.

From a system's design perspective, the modular architecture of the pipeline proved to be a key advantage. By maintaining distinct but interoperable components for ray tracing, visibility export, and geometry processing, the implementation remains flexible and extensible. This modularity makes it feasible to replace components independently. For instance, substituting OptiX with Vulkan RTX extensions, or libigl with an in-house mesh simplifier, without rewriting the entire system. This is crucial for real-world adoption, where developers must adapt algorithms to different engines, APIs, or hardware constraints. Furthermore, the use of open formats like glTF and OBJ ensures compatibility with mainstream tools and rendering engines, making the system suitable for integration into production workflows.

The paper also highlights the role of data interoperability in computational graphics. Transferring data between GPU and CPU domains, especially visibility buffers and triangle attributes, proved to be a central technical challenge. Careful design of data structures and serialization logic was necessary to avoid precision loss, maintain performance, and ensure synchronization between components.

Overcoming these challenges resulted in a pipeline that is not only functional but educational: it reveals the practical barriers between theoretical algorithms and their implementation in offline systems.

A notable lesson learned is that perceptual optimization often demands compromise. The perfect perceptual model would require continuous eye tracking, dynamic lighting estimation, and user-specific sensitivity calibration; an infeasible requirement in most contexts. The pipeline demonstrates that even a coarse perceptual model, grounded in visibility analysis, can significantly improve efficiency without compromising user experience. Achieving perceptual plausibility rather than perceptual perfection should guide future research in perceptually adaptive graphics.

Another outcome of this project is the recognition that perceptual approaches can enhance not only rendering but also the design process itself. By visualizing visibility heatmaps generated by libigl, designers and technical artists gain a new diagnostic tool for understanding how their models will be perceived. Such tools can inform artistic decisions, such as where to allocate polygon density, texture resolution, or shading detail. In this sense, perceptual decimation extends beyond optimization and becomes part of the creative feedback loop. Integrating perception-aware visualization into modeling software could empower artists to make data-driven aesthetic choices, fostering a more collaborative relationship between human intuition and computational analysis.

While the prototype developed here was evaluated primarily on static scenes, the underlying principles are generalizable to dynamic environments. In interactive applications such as games or simulations, where user viewpoint and lighting change continuously, a real-time extension of this system could operate as a background process, periodically updating visibility weights and adjusting level of detail (LOD) hierarchies accordingly.

Combining perceptual decimation with dynamic LOD management could enable systems that maintain consistent perceptual quality regardless of performance constraints. Such an approach would align closely with the goals of perceptual rendering, where computational resources are allocated in proportion to human attention rather than geometric uniformity.

Looking forward, several promising directions arise from this work. One is the incorporation of machine learning models that predict perceptual importance directly from rendered images or scene features. Neural networks trained on human eye-tracking data could infer saliency maps that augment or replace traditional visibility calculations. Another direction is temporal coherence, ensuring that perceptual simplifications remain stable over time to avoid distracting geometry popping. Additionally, integrating this system into augmented reality (AR) and virtual reality (VR) contexts would test its robustness under stereoscopic viewing and head-tracked rendering, where perceptual consistency is even more critical.

At a higher level, this research contributes to the ongoing dialogue between perception and computation. As rendering hardware grows exponentially faster and new ways of exploring 3D environments arise, the bottleneck of realism is no longer the number of polygons we can draw on screen but the efficiency with which we choose which polygons matter**.** In that sense, this project aligns with a broader paradigm shift toward systems that adapt to the limits and capabilities of human senses rather than merely pushing numerical boundaries.

In conclusion, the perceptual decimation system developed in this paper demonstrates that integrating visibility-based perceptual awareness into mesh simplification pipelines is both technically feasible and perceptually effective. By merging ray-traced visibility computation with geometry-aware decimation, the approach yields significant reductions in mesh complexity without compromising visual fidelity. The system bridges human perception and computational efficiency, offering a new lens through which future rendering and modeling pipelines can be designed. The hope is that this work inspires further exploration of perceptual intelligence in computer graphics, where algorithms not only compute but also see.

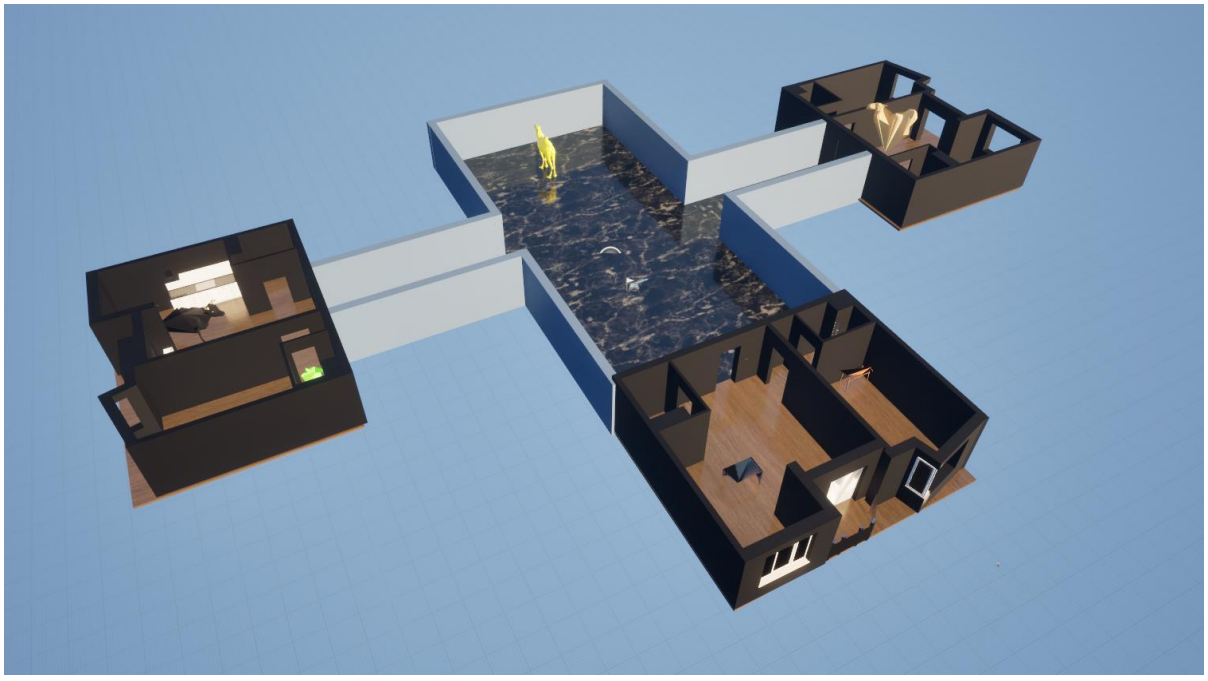


Figure 1.14: *Post edge-collapse decimation process - final decimated scene from free-camera POV.*

8. Definition of Terms

Acceleration Structure (BVH – Bounding Volume Hierarchy) *– A data structure used in ray tracing to accelerate intersection tests between rays and scene geometry. BVHs organize objects hierarchically into bounding volumes, allowing rays to skip large parts of the scene and significantly reduce computational overhead.*

Agent (Observer) *– A virtual entity or simulated camera that moves through the environment, representing either a player's movement path or a predefined camera trajectory. The agent's viewpoint is used to generate rays and collect visibility data.*

Camera Path / Viewpoint Sampling *– The process of defining positions and orientations from which rays are cast to estimate visibility. It simulates how different perspectives contribute to the overall importance distribution.*

Distance Weighting *– A method of assigning importance to triangles based on their distance from the observer, often using an inverse-square or logarithmic falloff to emphasize nearby geometry.*

Edge Collapse *– A fundamental operation in mesh simplification where two connected vertices are merged into one, reducing the number of faces while attempting to preserve the mesh's shape.*

Edge-Flap Data Structure *– A structure used in libigl to efficiently manage edges and adjacency information for edge-collapse simplification.*

Foveation / Foveated Weighting *– A method inspired by human vision that assigns higher importance to the central visual field (fovea) and lower weights to peripheral regions when analyzing visibility.*

Global Triangle Index *– A unique identifier for each triangle in the scene, ensuring consistent indexing across multiple meshes during visibility analysis.*

Heatmap Visualization *– A color-coded representation of importance or visibility values mapped onto a mesh surface. Warmer colors (e.g., red/yellow) indicate high importance, while cooler colors (e.g., blue) represent low importance.*

Importance Mapping *– The assignment of weights or scores to different regions of a mesh based on factors such as visibility, distance, or curvature, used to guide selective simplification.*

Least-Cost Edge Collapse *– A mesh simplification operation that merges the edge with the lowest computed cost, where cost is adjusted by geometric and importance-based factors.*

Level of Detail (LOD) *– Multiple versions of the same 3D model with varying polygon counts, used to optimize performance by displaying simpler versions when objects are far away from the viewer.*

libigl *– An open-source C++ geometry processing library that provides algorithms for mesh simplification, deformation, and analysis.*

Manifold Geometry *– A mesh property ensuring that each edge is shared by exactly two faces. This condition guarantees surface continuity and stable topological behavior during processing.*

Mesh Decimation *– The process of reducing the number of vertices, edges, and faces in a 3D mesh while preserving the overall shape and appearance of the model.*

Mesh Simplification Hierarchy *– The sequence of progressively reduced meshes generated by iterative simplification, often used to support LOD systems.*

Normalization *– The process of scaling data (such as visibility counts) into a standard range, typically between 0 and 1, to enable consistent interpretation across different datasets.*

Occlusion *– The blocking of one object by another in the line of sight from the observer's perspective. In visibility analysis, only the first visible surface hit by a ray is considered, while occluded surfaces are ignored.*

Offline Processing *– A computation approach where data is precomputed rather than generated in real time. The pipeline runs offline, producing optimized assets that can later be used in real-time applications.*

OptiX *– A high-performance GPU-accelerated ray tracing engine developed by NVIDIA, used in this study for visibility analysis.*

Payload (RadiancePRD) *– A data structure used in OptiX ray tracing to carry per-ray information such as accumulated weight, hit counts, or color values.*

Perceptual Fidelity *– The degree to which a simplified 3D model maintains the visual appearance and key features of the original high-resolution mesh as perceived by the viewer.*

Per-Face Weighting *– A process where each triangle (face) in a mesh is assigned a numeric weight based on importance, guiding how strongly it resists simplification.*

Ray Tracing *– A rendering technique that simulates the path of light rays in a virtual scene to compute visibility, shading, and reflections. In this study, it is used to determine triangle visibility.*

Ray Sampling Density *– The number of rays cast during visibility analysis. Higher sampling density improves accuracy but increases computational cost.*

Real-Time Mesh Decimation (Future Direction) *– A potential extension of the proposed system that would allow meshes to be dynamically simplified during runtime, adapting to camera movement or player actions.*

Static Scene Assumption *– A design constraint in which scene geometry and lighting remain unchanged during visibility analysis. This simplifies computation but limits applicability to dynamic environments.*

Unreal Engine 5 (UE5) *– A game engine developed by Epic Games, used in this study to create, view, and export 3D scenes for mesh decimation.*

Visibility Analysis *– The computational process of determining which parts of a 3D scene are visible from specific viewpoints. In this work, visibility is quantified through ray-triangle intersection tests.*

Visibility Votes *– The counts assigned to mesh triangles based on how frequently they are intersected by rays during visibility analysis.*

Weighted Decimation *– A simplification approach where different regions of a mesh are reduced according to assigned importance values, preserving high-importance areas and simplifying low-importance ones.*

References

1. libigl, a simple C++ geometry processing library, Alec Jacobson, Daniele Panozzo, Christian Schüller, Olga Diamanti, Qingnan Zhou, Sebastian Koch, Jeremie Dumas, Amir Vaxman, Nico Pietroni, Stefan Brugger, Kenshi Takayama, Wenzel Jakob, Nikolas De Giorgis, Luigi Rocca, Leonardo Sacht, Kevin Walliman, Olga Sorkine-Hornung, Teseo Schneider, and other, 2023.
https://libigl.github.io/

2. Ben Conway, "What Is Mesh Decimation and Why Is It Vital for AR?",
https://www.vntana.com/blog/what-is-mesh-decimation-and-why-is-it-vital-for-ar/

3. NVIDIA OptiX 9.0 – Programming Guide,
https://raytracing-docs.nvidia.com/optix9/guide/index.html

4. Evaluation of Illumination-Based Mesh Simplification, Xanthos Xanthopoulos, February 2021,
http://graphics.cs.aueb.gr/graphics/docs/Light_driven_mesh_simplification.pdf

5. Seamless: Seam erasure and seam-aware decoupling of shape from mesh resolution (Songrun Liu*, Zachary Ferguson*, Alec Jacobson, Yotam Gingold (*Joint first authors) | ACM Transactions on Graphics (TOG) 36(6),
Seamless: Seam erasure and seam-aware decoupling of shape from mesh resolution

6. Image-Driven Simplification, Peter Lindstrom Greg Turk, Georgia Institute of Technology
https://faculty.cc.gatech.edu/~turk/my_papers/image_simp_tog2000.pdf

7. Surface Simplification Using Quadric Error Metrics, Michael Garland, Paul S. Heckbert, SIGGRAPH 97
https://www.cs.cmu.edu/~garland/Papers/quadrics.pdf

8. Blender - The Free open Source 3D Creation Software,
https://www.blender.org/

9. glTF™ 2.0 Specification: The Khronos® 3D Formats Working Group version 2.0.1, 2021-10-11 23:01:57Z ,
https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

10. NVIDIA Developer Forums
https://forums.developer.nvidia.com/

11. glTF-Validator ,
https://github.khronos.org/glTF-Validator/

12. Sine-Cosine-Tangent | Glenn Research Center ,
https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/sine-cosine-tangent/

13. McGuire Computer Graphics Archive ,
https://casual-effects.com/data/

14. Blender Artists Community ,
https://blenderartists.org/

15. Alec Jacobson - Associate Professor of Computer Science , https://alecjacobson.com

16. Unreal Engine , https://www.unrealengine.com

17. Meshlab , https://www.meshlab.net/

18. Apartment floor mesh , https://www.cgtrader.com/free-3d-models/interior/house-interior/apartment-floor-plan-3d-model

19. Github, ClassicSponzaUnreal, radishface, 2024 https://github.com/radishface/ClassicSponzaUnreal

20. Github, GLTF-Renderer, droettger, lspanonv, 2024 https://github.com/NVIDIA/OptiX_Apps/blob/master/apps/GLTF_renderer/doc/README.md